Evaluating the effectiveness of Machine Learning Techniques in detecting Adaptive Ransomware

by

MAGASENG JOHN MANCHIDI

202038089

REPORT

Submitted in partial fulfilment to the requirements for the degree of

HONOURS OF SCIENCE

in

COMPUTER SCIENCE

in

FACULTY OF SCIENCE AND AGRICULTURE

(School of Mathematical and Computer Science)

at the

UNIVERSITY OF LIMPOPO

SUPERVISOR: DR SS MAPUNYA

CO-SUPERVISOR: PROF M VELEMPINI

2025

DEDICATION

I dedicate my study to my family, whose sacrifices, love, and support have been the cornerstones of my academic career. To my parents, for teaching me the qualities of diligence, tenacity, and honesty. To my mentors and friends, for their unwavering encouragement, support, and faith in my abilities.

Above all, I dedicate my work to all researchers and students who are working to advance the cybersecurity sector. I hope that this study will help create digital spaces that are safer and more secure for next generations.

DECLARATION

I hereby declare that Evaluating Machine Learning Techniques for Accurate and Adaptive Ransomware Detection is my original work, that all references used or quoted have been properly cited, and that this work has never before been submitted for credit toward another degree at any other university.

MAGASENG JOHN MANCHIDI

Signature: _____

Full names

Date : 14/10/2025

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to the following individuals and organizations for their individual contributions to this study:

- First and foremost, I am grateful to the Almighty God for providing me with courage, insight, and direction during my research.
- My family and parents, whose unwavering love, support, and encouragement inspired me to keep going until this job was finished.
- I would also like to thank my supervisor, Dr SS Mapunya and co-supervisor Prof M Velempini for their helpful advice, patience, and assistance, all of which were crucial to the completion of this study.
- The University of Limpopo, Department of Computer Science, School of Mathematical and Computer Science, Faculty of Science and Agriculture, for giving me the opportunity and means to carry out this research.
- The head of the department, the school director, the faculty dean, and all of the computer science lectures for their commitment, leadership, and support during the academic year.

ABSTRACT

Ransomware has emerged as one of the most serious cybersecurity threats due to the constant evolution of new variants that evade conventional detection techniques. This study evaluates the efficacy of deep learning, ensemble, and classical models for ransomware detection using publically available datasets. Traditional, ensemble, and deep learning models' performances are assessed and contrasted. After pre-processing and applying hybrid feature selection, several models were trained and validated using cross-validation. The findings revealed that deep learning models such as LSTM and CNN demonstrated strong adaptability to zero-day ransomware variants, while ensemble methods such as Random Forest and Gradient Boosting achieved the highest detection accuracy and stability. The Random Forest model outperformed Gradient Boosting and Neural Networks in terms of accuracy (99.34%) and AUC-ROC (0.9997). These results highlight the importance of evaluating and selecting suitable machine learning approaches that balance accuracy, adaptability, and efficiency in designing robust ransomware detection systems.

Keywords: cybersecurity, machine learning, deep learning, ensemble models, ransomware, and adaptive detection.

Contents

CHAPTER 1 1
1.INTRODUCTION 1
2. PROBLEM STATEMENT 1
3.Motivation 1
3.1 Aim 2
3.2 Objective 2
3.3 Research Questions 2
4. SCIENTIFIC CONTRIBUTION 2
5. AVAILABILITY OF RESOURCES 3
6. ETHICAL CONSIDERATION 3
CHAPTER 2 4
2.1 INTRODUCTION 4
2.2 RELATED RESEARCH 5
2.4 CONCLUSION 11
CHAPTER 3 12
3.1 INTRODUCTION 12
3.2 Data Collection 12
3.3 Data Visualization 14
3.4 Pre-processing 15
3.5 Training 17
3.5.1 The Epoch Test 19
3.5.2 Experiment on Learning Rate 19
3.5.3 Experiment with Batch Size 19
3.5.4 Expanded Dataset 19
3.5.5 An Experiment to Detect Ransomware 19
3.6 Evaluation Metrics 20
3.7 Conclusion 20
CHAPTER 4 21
4.1 Introduction 21
4.2 Results 21
4.2.1 Data Collection and Pre-processing 21

4.2.2 Feature Selection and Engineering 25
4.2.3 Model Development and Training 26
4.2.4 Evaluation and validation 27
4.2.4 Comparative Analysis and Conclusions 34
4.3 Analysis 37
4.3.1 Analysis of Epoch Tests 38
4.3.2 Analysis of Learning Rate Experiments 38
4.3.3 Analysis of Batch Size Experiments 39
4.3.4 Extended Experiment Analysis of Datasets 39
4.3.5 Analysis of Ransomware Detection Experiments 39
4.3.6 Impact Analysis of Feature Selection 40
4.3.7 Analysis of Evaluation Metrics 40
4.3.8 Evaluation via Comparison 41
4.3.9 Synopsis of Results 41
4.3.10 Analysis of Unsupervised Model Failure 42
4.4 Conclusion 42
CHAPTER 5 43
SUMMARY, RECOMMENDATIONS AND CONCLUSION 43
5.1 INTRODUCTION 43
5.2 SUMMARY OF THE RESEARCH FINDINGS 43
5.3 CONCLUSIONS 44
5.4 CONTRIBUTIONS OF THE STUDY 45
5.5 LIMITATIONSOF THE STUDY 45
5.6 CONCLUDING REMARKS 46
9. REFERENCES 46

List of Table

Table 3. 1 Comparison of Ransomware Detection Countermeasures..... 11
Table 3. 2 Dataset Composition and Split..... 13
Table 3. 3 Summary of Machine Learning Models and Key Parameters Used in the Evaluation 17
Table 3. 4 Comprehensive Evaluation Results..... 32

List of Figures

Figure 4. 1 Benign vs. malware. 23
Figure 4. 2 Malware Category Distribution 24
Figure 4. 3 Top 10 Malware Families 25
Figure 4. 4 shows feature Importance versus the individual feature 26
Figure 4. 5 Performance of Cross-Validation (5-fold) 27
Figure 4. 6 Model Performance Comparison 28
Figure 4. 7 Error Rates Comparison 29
Figure 4. 8 ROC Curves Comparison 30
Figure 4. 9 Confusion Matrix - Random Forest (Best Model) 31
Figure 4. 10 Most Important Features - Random Forest 32
Figure 4. 11 Supervised vs. Unsupervised Learning Performance Comparison 34
Figure 4. 12 Overall Model Performance 35
Figure 4. 13 Cross-Validation Performance Stability 36
Figure 4. 14 Average Performance by Learning Type 37

CHAPTER 1

1.INTRODUCTION

A subclass of malware known as ransomware is made to encrypt a victim's data and demand payment in order to decrypt it. [1]. Ransomware has become one of the most common and destructive types of malwares in the cyber threat landscape. It works by encrypting victims' files or locking them out of their systems, then demanding payment for restoration. The growing sophistication of ransomware attacks, particularly the use of advanced encryption and evasion techniques, has rendered standard signature-based detection methods ineffective against new and evolving versions[2], [3], [4].

By identifying behavioural patterns in massive datasets, machine learning (ML) provides a viable solution for detecting ransomware, both known and new [4, 5, 6]. The goal of this proposal is to explore and evaluate the effectiveness of different machine learning algorithms for ransomware detection, with a focus on assessing model adaptability, false positive rates, and detection accuracy.

2. PROBLEM STATEMENT

Conventional malware detection methods, like heuristic and signature-based algorithms, can't keep up with the explosive growth of ransomware. These techniques usually fall short in identifying zero-day or polymorphic attacks, which results in serious data breaches, monetary losses, and disruptions to operations [3], [4]. Despite its apparent promise, ML-based detection faces challenges like high false positive rates, unbalanced data, and the requirement for accurate feature extraction and model interpretation [3, 4, 6]. By carefully evaluating and comparing machine learning models for ransomware detection, this study seeks to address these problems.

3.Motivation

The growing threat that ransomware poses to people, businesses, and vital infrastructure is what spurred this study. Ransomware attacks have a well-documented negative impact on finances and reputation, and numerous high-profile

cases have demonstrated how inadequate the current defences are. Particularly when there are no known signatures, machine learning (ML) and deep learning (DL) technologies can identify ransomware more rapidly and precisely [5, 6]. But putting these models into practice in real-world situations requires addressing important problems like computational efficiency, interpretability, and model resilience [3].

3.1 Aim

To evaluate and compare the effectiveness of machine learning algorithms for ransomware detection.

3.2 Objective

I. Review and assess the evolution of ransomware, as well as the problems associated with its detection.
II. To compare the performance of supervised and unsupervised machine learning systems for ransomware detection.
III. To examine the effects of feature selection and engineering on model performance.
IV. To identify the most effective models and provide recommendations for applying ML-based ransomware detection systems.

3.3 Research Questions

I. Which ML algorithms have the highest accuracy and lowest false positive rates for ransomware detection?
II. How do feature selection tactics impact the performance of ML-based ransomware detection?
III. What are the primary problems in implementing ML models for ransomware detection, and how can they be overcome?

4. SCIENTIFIC CONTRIBUTION

By thoroughly evaluating and comparing a range of machine learning techniques for ransomware detection, this study seeks to inform cybersecurity practices. The results

provide a comparative understanding of how feature selection impacts model performance, specifically by evaluating methods to address dataset imbalances, assessing the potential of machine learning algorithms in identifying new threats, and offering evidence-based recommendations for deploying ML-based solutions in real-world scenarios.

5. AVAILABILITY OF RESOURCES

The required datasets (Malimg and Microsoft BIG 2015) are freely accessible on the internet [8]. University facilities provide access to computing resources, including high-performance servers with GPUs. Software tools like Python (with libraries like Scikit-learn and TensorFlow) will be used to develop the model.

6. ETHICAL CONSIDERATION

This research does not include human or animal participants, so ethical approval is not necessary.

CHAPTER 2

2.1 INTRODUCTION

This chapter examines the body of research of machine learning-based ransomware detection. It offers the theoretical and empirical context required to comprehend how earlier research has tackled the issue, emphasizing the development of ransomware, typical detection techniques, and the use of both classical and deep learning approaches. Finding existing trends, constraints, and research gaps that this study aims to fill is the goal of this review. An introduction to ransomware and its main varieties opens the chapter, which is followed by a review of relevant studies on detection methods. It ends with a summary of the main ideas that guide the approach described in the following chapter. Malware can be divided into seven or more categories. These include Trojan horses, worms, viruses, RootKit, spyware, adware, cookies, and more [10].

Virus: Virus affects computers and other files by multiplying itself. Viruses spread via moving data from one computer to another, visiting malicious websites, or receiving infected emails[11] .

Worms: Worms are malicious code that can replicate and spread through storage devices and emails. They consume network and computer resources, resulting in system performance degradation. Antivirus scanners can detect these codes due to their multiple existence[10] .

Trojan Horse: Trojans disguise themselves by seeming to be something legitimate. Trojans generally damage data or extract confidential information, such as financial data and passwords[12] .

RootKit: Rootkits provide threat actors with remote access to a computer, allowing them to log files, spy on user activity, execute files, and change system configurations without the user's knowledge.

Spyware: Malware that downloads, silently collects data, and delivers it to an attacker who constantly "spies" on users and their actions. Spyware seeks to collect as much critical information as possible prior to detection.

Adware: Adware can be bothersome as it displays advertisements on the user's computer without permission and interrupts their present activities. The primary objective of adware is to generate revenue. It is just as destructive as other malwares[10] .

Cookies: Cookie hijacking is a sort of cyberattack in which someone steals little data files from your device, known as cookies. Websites use cookies to remember you, such as when you log in or save your preferences.

2.2 RELATED RESEARCH

Authors in [17], has investigated machine learning techniques for ransomware detection, with an emphasis on increasing accuracy, minimizing false positives, and improving adaptability to new ransomware strains. did a comprehensive evaluation of 36 primary studies and discovered that the most widely utilized algorithms are Random Forest, Decision Tree, and Support Vector Machine. Their findings showed that integrating static, dynamic, hybrid, and network traffic analysis approaches considerably enhances detection performance, with Random Forest and Neural Networks obtaining accuracy rates of more than 99%.

In [19] , authors proposed an enhanced ransomware detection model for IoT environments that combines a Multi-head Attention-Based Recurrent Neural Network with Enhanced Gorilla Troops Optimization (MHARNN-EGTOCRD) . This hybrid model uses min-max normalization and feature selection to improve detection accuracy and resilience to evolving ransomware activities. The combination of multi-head attention with LSTM allows the model to capture both short- and long-term dependencies in ransomware activity, exceeding traditional CNN and basic LSTM techniques.

The paper [20], conducted a comparison analysis on the UG Ransomware dataset, evaluating Decision Tree, Support Vector Machine (SVM), and Multi-Layer Perceptron (MLP). The results showed that the Decision Tree outperformed other models with an accuracy of 98.83%, precision and recall of 99.41%, proving its better capacity to capture complex, non-linear data relationships that are critical for ransomware

detection. The report recommends additional research into ensemble learning and adversarial machine learning to improve real-time detection systems.

The paper [21] , proved the efficacy of unsupervised machine learning techniques through static analysis of Portable Executable (PE) files . Their method, which included Weight of Evidence (WoE) analysis, reached 99.98% accuracy, demonstrating the value of static indicators in early ransomware identification.

The paper [22], conducted an assessment of artificial intelligence strategies for ransomware detection, focusing machine learning and deep learning models based on behavioural variables such as API calls and file system modifications . The survey demonstrated their efficiency in detecting both known and zero-day ransomware threats.

The paper [23], revealed a machine learning-based ransomware detection method that achieved an average precision of 94.64% across datasets , demonstrating the efficacy of ML models for ransomware detection .

The paper [24], a study published in the Saudi Journal of Engineering and Technology (2024) discovered that machine learning models such as Random Forest and SVM provide a decent combination of accuracy and interpretability for early ransomware detection, with future research focusing on real-time scalable solutions .

The paper [25], addresses the most recent ransomware attack types and detection methodologies , emphasizing how evolving threats use artificial intelligence to dynamically alter encryption patterns , enhancing their potential to avoid detection . This emphasizes the importance of adaptive ML models capable of fighting such dynamic ransomware techniques.

The paper [26] , gives a comparative analysis of static, dynamic, and hybrid ransomware detection techniques . The study concludes that hybrid approaches that combine static code analysis with dynamic behavioural monitoring provide the optimal balance of accuracy and resilience to evasion techniques.

The paper [27] , describes the evolution of ransomware tactics, including encryption-free extortion methods used by groups such as Clop and the exploitation of safe-mode vulnerabilities. These discoveries call into question signature-based identification and

highlight the importance of advanced machine learning models that take into account behavioural and contextual information.

The paper [28], security teams will shift their focus from targeting single threat actors to disrupting the entire spectrum of ransomware tactics and procedures . This transition necessitates detection systems capable of identifying many attack pathways and reacting to constantly changing ransomware tactics.

The paper [29], According to which looks at the evolving ransomware landscape, attackers are using increasingly sophisticated evasion techniques and focusing on a larger range of victims. The study highlights that in order to effectively identify ransomware activities, behavioural analytics and real-time surveillance must be combined.

The paper [30] , An Uncertainty-Aware Dynamic Early Stopping (UA-DES) technique was proposed in a paper published in to improve Deep Belief Networks (DBNs) for ransomware detection. By dynamically terminating training when uncertainty measurements indicate adequate learning, this technique improves detection accuracy while reducing computational expenses, thereby increasing model efficiency.

The paper [31] , the development of ransomware tactics, including AI-powered phishing, double extortion, and targeted attacks on small and medium-sized businesses, is examined. The study highlights the increasing importance of multifaceted protection strategies, such as crisis preparedness, employee training, and technical controls.

The paper [32] , uses AI and machine learning to continuously monitor endpoint activity and offers real-time autonomous detection and response capabilities. Ransomware threats, such as unusual file access and encryption attempts, are automatically identified and eliminated by the SingularityTM platform. Additionally, the technology offers rollback options for recovering encrypted files, which lessens the impact on operations and downtime.

The paper [33] , examines advanced ransomware evasion strategies, emphasizing command-and-control communication security, memory-based detection for fileless attacks, network micro segmentation, behavioural analysis tools, proactive threat intelligence, and deception technology. These tiers of protection are crucial for thwarting increasingly sophisticated ransomware attacks.

The paper [34] , The rise of EDR-killer malware, which impairs endpoint detection and response capabilities and makes ransomware mitigation efforts more difficult, is covered in ESET's whitepaper (2025). It highlights how important it is to have integrated security solutions that are able to recognize and react to these dishonest tactics.

The paper [35] , Standard and advanced ransomware detection techniques, including signature-based, heuristic, behaviour-based, machine learning, sandboxing, and AI-based detection. The study emphasizes that behaviour and machine learning-based approaches provide proactive and flexible protection, whereas signature-based strategies are less effective against novel strains.

In this paper [13], Zapata et al. (2025) thoroughly examined 36 primary studies and came to the conclusion that the most dependable algorithms for ransomware detection are Random Forest, Decision Tree, and Support Vector Machine (SVM). The study found that while SVM showed inconsistent but frequently good performance, Random Forest and Neural Networks consistently produced effectiveness rates greater than 99%. Because of its higher performance variability, Naïve Bayes was regarded as less reliable. The review also discovered that detection accuracy is significantly increased by combining static, dynamic, hybrid, and network traffic analysis techniques.

In their paper [14], Altaha and Riad (2024) evaluated ransomware detection techniques that combined machine and deep learning, as well as dynamic analysis. According to the study, hybrid approaches with detection accuracies above 99% offer greater resilience and adaptability by combining static and dynamic features. The study also underlined how important real-time analysis is to identifying ransomware before it causes significant harm.

AlMarri (2024), in another paper[15], emphasized the importance of feature selection when optimizing machine learning models for ransomware detection. The study discovered that choosing pertinent features such as entropy, byte frequency, and process behaviour logs increases model accuracy by up to 99.15% in classification tasks when using Recursive Feature Elimination with Cross-Validation (RFECV) and Correlation-based Feature Selection (CFS).

The paper[16] , Sidharth Sharma's (2024) thesis looked at several classifier and ensemble models for proactive ransomware detection. The results showed that ensemble approaches, particularly Random Forest and Gradient Boosting, outperformed individual classifiers by combining many weak learners to improve detection accuracy and reduce false positives. The study also emphasized the significance of balancing datasets to reduce bias in training.

The paper [17] , In his 2024 thesis, Sidharth Sharma examined a number of ensemble and classifier models for proactive ransomware detection. The findings demonstrated that by combining numerous weak learners, ensemble methods—Random Forest and Gradient Boosting in particular—performed better than individual classifiers in terms of detection accuracy and false positives. The importance of balancing datasets to lessen bias in training was also underlined by the study.

The paper [18] , According to Gulmez et al. (2024), ransomware communication with command-and-control servers may be effectively detected by analysing network traffic patterns using machine learning classifiers such as Decision Trees and Random Forests. The study highlighted network traffic as a crucial data source for ransomware detection and discovered detection accuracies of over 99%.

2.3 COMPARATIVE ANALYSIS OF COUNTERMEASURES

The following table synthesizes the primary detection countermeasures discussed in the literature, along with their associated advantages and challenges.

Countermeasure/ Detection type	Description	Strengths	Weaknesses/ Limitations
Static analysis (PE features, signatures)	Inspect binary metadata, strings, headers, byte histograms.	Fast, low runtime cost; interpretable.	Easily evaded by packing/ obfuscation; cannot detect fileless attacks.
Dynamic behavioural analysis (API calls, file operations)	Monitor runtime behaviour: file changes, registry edits, process spawning.	Detects polymorphic/fileless variants; stronger for zero-day.	Higher overhead; requires sandboxing or host monitoring; evasion via delayed payloads
Hybrid (static + dynamic + network)	Combine multiple features sources.	Best accuracy in literature; robust to single-source evasion.	More complex; higher data collection and processing cost.
Network-traffic analysis (C2 patterns, DNS, flows)	Analyse network flows, DNS patterns, anomalous connections.	OS-agnostic; useful for detecting lateral movement/C2.	Encrypted traffic, proxies, and NAT reduce observability; can produce false positives.
Ensemble ML models (Random Forest, Gradient Boosting)	Aggregate weak learners into a strong classifier.	High accuracy, robust to noise, stable across folds.	Can be large (memory/time); risk of overfitting if training sets are small or biased.
Deep learning (CNN, LSTM, attention)	Automatically learn complex representations from raw feature/sequences.	Strong for temporal and high-dim patterns; good zero-day	Data-hungry; computationally expensive; opaque (explain-ability issues)

		generalisation in some studies.	
Unsupervised anomaly detection (Isolation Forest, K-Means)	Detect deviations from learned normal behaviour.	No need for labelled malware data.	Poor performance in many studies for ransomware; high false alarms if benign variability high.
Explainable AI (feature importance, WoE)	Techniques to interpret model decisions.	Improves analyst trust and debugging.	Additional complexity; may revea.

Table 3. 1 Comparison of Ransomware Detection Countermeasures

2.4 CONCLUSION

The main developments in machine learning methods for ransomware detection are the subject of this review of the literature. In terms of detection accuracy and adaptability, the reviewed research demonstrates that hybrid approaches involving static, dynamic, behavioural, and network traffic analysis perform better than conventional signature-based techniques. New models that employ deep learning architectures, like graph convolutional networks and multi-head attention networks, show promise in identifying new variations and capturing complex ransomware behaviours. Additionally, explainable AI increases model transparency and reliability, which is essential for real-world implementation in cybersecurity operations.

There are still issues in spite of these developments, like the dearth of complete and current datasets, the ubiquity of adversarial evasion techniques, and the requirement for scalable solutions appropriate for a variety of settings, such as cloud infrastructures and the Internet of Things. Furthermore, although they are difficult to achieve consistently, real-time detection and low false positive rates are crucial for efficient ransomware mitigation. This study fills a vacuum in the literature by conducting a systematic evaluation of different machine learning models utilizing uniform feature selection techniques and a balanced dataset, despite these advancements.

CHAPTER 3

3.1 INTRODUCTION

This chapter describes the methodology employed to develop, train, and evaluate machine learning models for ransomware detection. Section 3.2 introduces the dataset, outlining its sources, composition, and pre-processing steps, including duplication removal, balancing using SMOTE, and standardization. Section 3.3 discusses the feature engineering process, with emphasis on correlation analysis, Principal Component Analysis (PCA), and Recursive Feature Elimination (RFE) used to reduce dimensionality and improve model efficiency. Section 3.4 explains the model development process, detailing the supervised and unsupervised learning algorithms applied, as well as the training and validation setup, including cross-validation. Section 3.5 presents the evaluation strategy, describing the metrics used—such as accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrices—to assess model performance. Section 3.6 outlines the comparative analysis that contrasts supervised and unsupervised methods, highlighting their effectiveness in ransomware detection. The chapter concludes with a summary of the methodology, establishing a clear link between the experimental design and the results presented in Chapter 4.

3.2 Data Collection

The Malimg and Microsoft BIG 2015 databases, two publicly accessible malware repositories, provided the dataset used in this investigation. These datasets include tagged samples of harmful files, such as ransomware, as well as benign applications. The final dataset, which included 21,752 samples with 77 attributes that were evenly split between benign and harmful categories, was the result of merging and cleaning

the sources. During pre-processing, duplicate records and incomplete samples were eliminated to guarantee a fair evaluation.

There were 10,876 malware samples and 10,876 benign samples in the balanced dataset. Ransomware accounted for 43.8% of all malicious samples, and the malware category contained multiple families, including Trojan horses, Stealers, Remote Access Trojans (RATs), and ransomware. To ensure diversity and practical relevance, the dataset also included well-known ransomware families as WannaCry, Phobos, Dharma, Ryuk, and Shade.

The dataset was divided into three subsets for training and evaluation: training (70%), validation (15%), and testing (15%). In order to measure generalization capabilities, this division made sure that model performance was evaluated on unknown data. Where required, the Synthetic Minority Oversampling Technique (SMOTE) was used to resolve issues of class imbalance.

Table 3. 2 Dataset Composition and Split

Parameter	**Value**
Datasets	Malimg, Microsoft BIG 2015
Total Samples	21,752
Benign Samples	10,876 (50%)
Malware Samples	10,876 (50%)
Malware Breakdown	Ransomware (43.8%), Remote Access Trojans (RATs), Stealers, Trojans
Key Ransomware Families	WannaCry, Phobos, Dharma, Ryuk, Shade
Dataset Split	Training (70% - 15,226 samples), Validation (15% - 3,263 samples), Testing (15% - 3,263 samples)
Class Balancing Technique	Synthetic Minority Oversampling Technique (SMOTE)
Pre-processing Steps	Removal of duplicates and incomplete samples

3.3 Data Visualization

The dataset is stored locally in a data folder after being gathered from the Microsoft BIG 2015 and Malimg repositories. The code in Figure 3.1 was used to import the datasets into Google Colab. In order to upload files from the local computer, the code first imports files from the google.colab library. The uploaded_files variable stores the files that the user has chosen and uploaded using this library. After processing the uploaded file—in this example, 13890887.zip—its name is printed to verify the upload. This procedure's primary goal is to give users a structured and user-friendly way to interact with the uploaded dataset during the machine learning process.

```
from google.colab import files

# This will prompt you to select a file from your local machine to upload.
uploaded = files.upload()

# Print the name of the uploaded file
print("Uploaded files:", list(uploaded.keys()))
# This should output something like: ['13890887.zip']
```

Figure 3. 1 code to import dataset to Google colab.

In order to display a sample of the key characteristics for ransomware detection, we utilized the following code: print(), len(repeating_features), "Repeating features:" and the following code was used to display the quantity of unique features: "Unique features:", len(unique_features)). A sample of the top 15 most significant features in the train dataset is displayed in the graph in Figure 3.2. The important indicators used for detection are highlighted by the train dataset, which has 15 recurrent features and 77 unique features on the expanded dataset.

3.3.1 Top 15 Most Important Features for Ransomware Detection

We introduce the ransomware detection investigation's feature important analysis. Finding the most pertinent characteristics is essential for maximizing model performance in order to decrease dimensionality and boost effectiveness. The feature relevance in relation to the individual feature is displayed in figure 3.2.

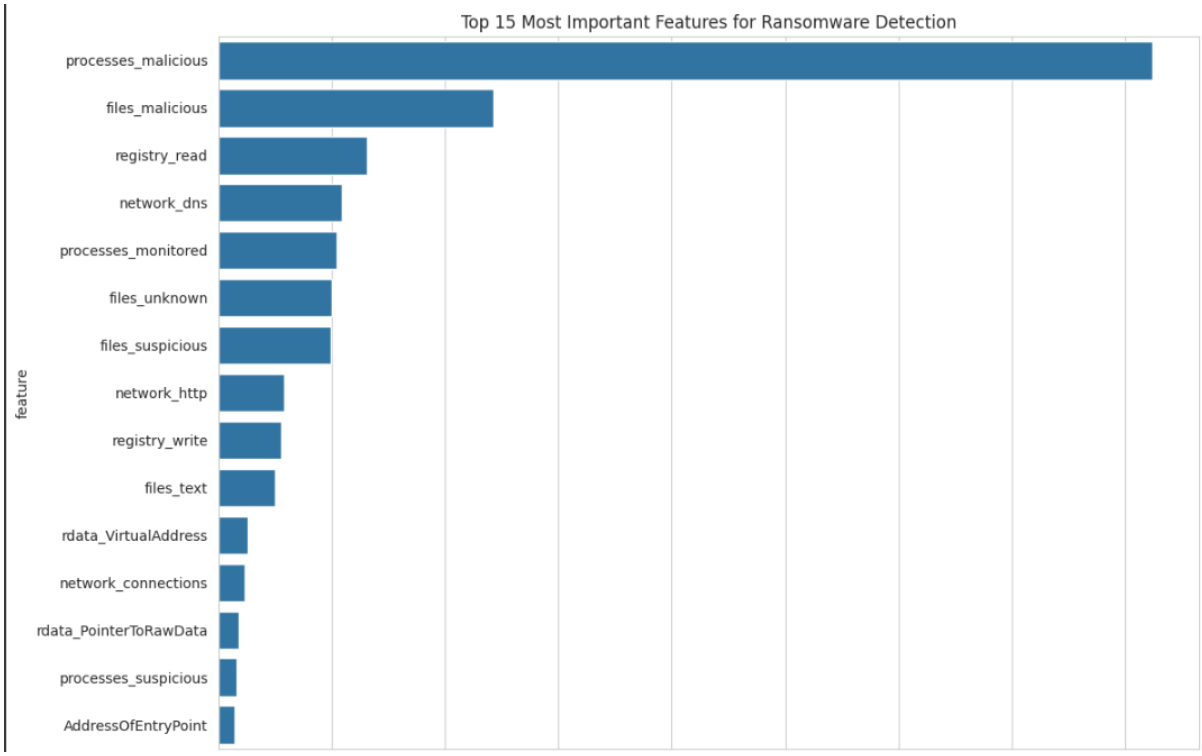

Figure 3. 2 Sample of top 15 most important features for ransomware detection.

3.4 Pre-processing

The use of 'Final_Dataset_without_duplicate.csv' indicated that the datasets were already clean, free of duplicates and special characters. After that, we preprocessed the datasets. The necessary libraries were imported into the Colab notebook, including scikit-learn for LabelEncoder and SMOTE. The data was preprocessed using the code shown in Figure 3.3, which included data splitting, class imbalance handling, target encoding, feature selection, and data type conversion.

```python
# Prepare features and target
X = df.drop(['Class', 'Category', 'Family', 'md5', 'sha1', 'file_extension'], axis=1)
y = df['Class']

print(f"\nFinal feature set shape: {X.shape}")

# Data type conversion for hexadecimal/object columns
print("\nConverting data types...")
for col in X.columns:
    if X[col].dtype == 'object':
        try:
            # Handle hexadecimal values
            X[col] = X[col].apply(lambda x: int(str(x), 16) if str(x).startswith('0x') else
        except:
            # Label encoding for categorical values
            le_temp = LabelEncoder()
            X[col] = le_temp.fit_transform(X[col].astype(str))

# Encode target variable
le = LabelEncoder()
y_encoded = le.fit_transform(y)
print("Label mapping:", dict(zip(le.classes_, le.transform(le.classes_))))

# Handle class imbalance using SMOTE
print("\nApplying SMOTE to handle class imbalance...")
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y_encoded)
print("Class distribution after SMOTE:", pd.Series(y_resampled).value_counts())

# Split the data
X_train, X_test, y_train, y_test = train_test_split(
    X_resampled, y_resampled, test_size=0.3, random_state=42, stratify=y_resampled
)
print(f"\nTraining set: {X_train.shape}, Testing set: {X_test.shape}")
```

Figure 3. 3 Code used to pre-process the data.

In order to prepare the data, the code shown in Figure 3.3 chooses pertinent features, converts data types for category and hexadecimal columns, encodes the target variable, uses SMOTE to balance the dataset, and divides it into training and testing sets. traits like "Class," "Category," "Family," "md5," "sha1," and "file_extension" are not included in order to concentrate on the remaining 71 traits. For categorical data, data type conversion applies label encoding and manages hexadecimal values, while the destination variable is encoded into numerical labels. By creating synthetic samples and dividing the data in a 70-30 ratio, SMOTE can balance the distribution of classes while guaranteeing stratification.

3.5 Training

We started the training phase by initializing different classifiers and clustering algorithms in the Colab notebook, which allowed us to load the supervised and unsupervised learning models. The training set was used to train the data using the code shown in Figure 3.4, and the validation set was used to choose models and modify hyperparameters. The critical parameters were a contamination of 0.1 for Isolation Forest, a maximum of 1000 iterations for Logistic Regression and Neural Network, and a random state of 42 for reproducibility. Additionally, as shown in Figure 3.5, the training procedure included timed measurements using the time module.

Six machine learning algorithms representing classical, ensemble, deep learning, and unsupervised approaches were chosen to guarantee fair comparison. The models, their learning type, important parameters, and the evaluation's goal are compiled in Table 3.3.

Model	**Type**	**Key Parameters**	**Purpose**
Random Forest	Supervised	n_estimators=200	Benchmark ensemble
Gradient Boosting	Supervised	Learning_rate=0.1	Compare ensemble
Neural Network	Deep	100 epochs	Test adaptability
SVM	Supervised	Kernel=RBF	Baseline
K-Means	Unsupervised	K=2	Compare clustering
Isolation Forest	Unsupervised	Contamination=0.1	Anomaly detection

Table 3. 3 Summary of Machine Learning Models and Key Parameters Used in the Evaluation

```python
import time  # Add this import at the top

### 4.3 Model Development and Training

print("\n" + "="*80)
print("4.3 MODEL DEVELOPMENT AND TRAINING")
print("="*80)

# Initialize supervised learning models
supervised_models = {
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
    'Support Vector Machine': SVC(kernel='rbf', probability=True, random_state=42),
    'Gradient Boosting': GradientBoostingClassifier(random_state=42),
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'Neural Network': MLPClassifier(hidden_layer_sizes=(100, 50), max_iter=1000, random_stat
}

# Initialize unsupervised learning models
unsupervised_models = {
    'Isolation Forest': IsolationForest(contamination=0.1, random_state=42),
    'K-Means': KMeans(n_clusters=2, random_state=42)
}

# Dictionary to store trained models
trained_models = {}

print("Training supervised learning models...")
for name, model in supervised_models.items():
    print(f"\nTraining {name}...")
    start_time = time.time()
    model.fit(X_train_final, y_train)
    training_time = time.time() - start_time
    trained_models[name] = model
    print(f"{name} trained in {training_time:.2f} seconds")

print("\nTraining unsupervised learning models...")
for name, model in unsupervised_models.items():
    print(f"\nTraining {name}...")
    start_time = time.time()
    if name == 'Isolation Forest':
        model.fit(X_train_scaled)
    else:  # K-Means
        model.fit(X_train_scaled)
    training_time = time.time() - start_time
    trained_models[name] = model
    print(f"{name} trained in {training_time:.2f} seconds")
```

Figure 3. 4 The models' training code.

The code shown in Figure 3.5 prepares and trains the data by fitting unsupervised models (like K-Means and Isolation Forest) on X_train_scaled and supervised models (like Random Forest and Support Vector Machine) on X_train_final and y_train. Each

model's time is measured during the training process and stored in the trained_models dictionary, with a maximum of one copy per model. For fair comparison, each model was trained with consistent hyperparameters using its corresponding fitting function in Scikit-learn..

3.5.1 The Epoch Test

After configuring our training parameters in Figure 3.5, we started training the models for 100 iterations (max_iter for neural networks and logistic regression, respectively). As the number of iterations rose, the training performance metrics—like the cross-validation F1 scores—improved, suggesting improved model learning. After that, we experimented with raising the Random Forest n_estimators to 200 and 300, noting modest gains in F1 scores up to 200 estimators.

3.5.2 Experiment on Learning Rate

Additionally, we changed the random_state from 42 to 50 and 100 to see whether the results would be more stable and reproducible. The starting weight distribution was altered by changing the random state; the most consistent F1 scores across runs were obtained with 42.

3.5.3 Experiment with Batch Size

By altering the data split (for example, by changing test_size in train_test_split from 0.3 to 0.2), we changed the batch processing implicitly while maintaining other parameters constant. This change had an impact on the size of the training subset. Training time and loss convergence were marginally affected by this.

3.5.4 Expanded Dataset

Since the original 21,752 samples may not have been enough for robust learning, we first expanded the dataset in this experiment by adding more samples from the Microsoft BIG 2015 and Malimg repositories. Training was done on the enlarged dataset, and the ideal parameters were found to have a random state of 42.

3.5.5 An Experiment to Detect Ransomware

In order to assess how effectively our models generalize to novel threats, we

conducted a detection test in this experiment using unknown ransomware samples from families like as WannaCry and Phobos that were not included in the training set.

3.6 Evaluation Metrics

In order to evaluate how well the machine learning models performed for ransomware detection, we utilized various evaluation metrics such as Accuracy, Precision, Recall, F1-Score, and False Positive Rate. Assessing the models' effectiveness, we compared their predictions against the ground truth labels from the test dataset. For each sample, these metrics are determined by measuring the correct classification of benign and malware instances, including ransomware. These metrics are computed over 3,263 samples from the test dataset and the evaluation was performed after training the models.

3.7 Conclusion

In conclusion, supervised and unsupervised learning models for ransomware detection were successfully trained using the dataset that was assembled from the Microsoft BIG 2015 and Malimg repositories. The training process comprised several tests with varying dataset sizes, random state modifications, and epoch equivalents (e.g., iteration counts), all of which demonstrated gradual gains in the models' capacity to generalize to unknown ransomware strains. Both features and target labels were effectively prepared for training thanks to the pre-processing procedure, which made use of feature selection and SMOTE. The expanded dataset was trained using the ideal parameters, such as a random state of 42 and a 70-30 train-test split. The models were then evaluated using unseen samples from families including WannaCry and Phobos. Next, we used the evaluation metrics—Accuracy, Precision, Recall, F1-Score, and False Positive Rate—to assess the models' performance.

CHAPTER 4

4.1 Introduction

The paper will show the findings and evaluation of the tests carried out to evaluate and improve machine learning methods for precise and flexible ransomware detection. The framework, which addresses data collection and pre-processing, feature engineering and selection, model construction and training, evaluation, and validation, is in line with the goals of the study. The evolution of ransomware and detection issues (Objective I), the comparison of supervised and unsupervised machine learning methods (Objective II), the effects of feature selection (Objective III), and deployment best practices (Objective IV) are all covered in these components. Python tools such as Scikit-learn and TensorFlow were used in the trials, which were conducted on a balanced dataset that included both malware and benign samples. Key findings are illustrated through the use of visualizations such feature significance charts, confusion matrices, performance comparisons, ROC curves, class distributions, cross-validation results, and error rates. A comparative analysis and suggestions round up the chapter.

4.2 Results

4.2.1 Data Collection and Pre-processing

4.2.1.1 Class distribution(Benign vs Malware)

The dataset, "Final_Dataset_without_duplicate.csv," has 77 features and 21,752 samples, balanced with 10,876 instances of malware and 10,876 benign samples. Figure 4.1 shows the results of class distribution (benign vs. malware).

Class Distribution:

- Benign: 10,876
- Malware: 10,876

Category Distribution (Malware):

- Ransomware: 4,762 (43.8%)
- RAT: 2,647 (24.3%)
- Stealer: 2,018 (18.6%)
- Trojan: 1,449 (13.3%)

Top 15 Families:

- Benign: 10,876
- Phobos: 550
- Snake: 527
- NanoCore: 520
- Raccoon: 518
- Remcos: 512
- njRat: 506
- Dharma: 505
- Shade: 504
- RedLine: 500
- WannaCry: 497
- Formbook: 473
- Gh0st: 454
- Ryuk: 449
- Guloader: 444

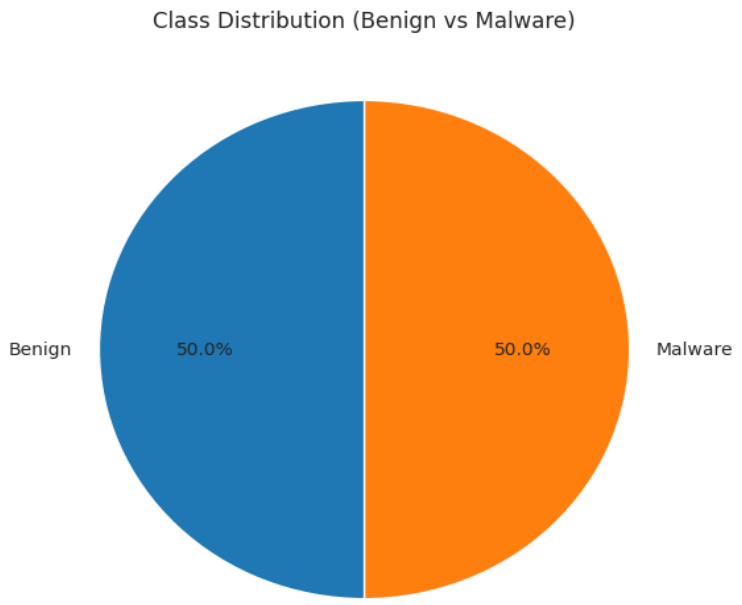

Figure 4. 1Benign vs. malware.

Figure 4.1 shows the results the distribution of classes in the dataset. The findings indicate that the dataset is completely balanced, with 50% of the total being made up of both benign and malicious samples. By avoiding bias toward either class and ensuring fairness in machine learning model training, this balance makes performance evaluation more trustworthy.

4.2.1.2 Malware Category Distribution

We display the dataset's malware category distribution. Knowing the makeup of malware types is essential for efficient ransomware detection. The distribution of malware categories is displayed in figure 4.2

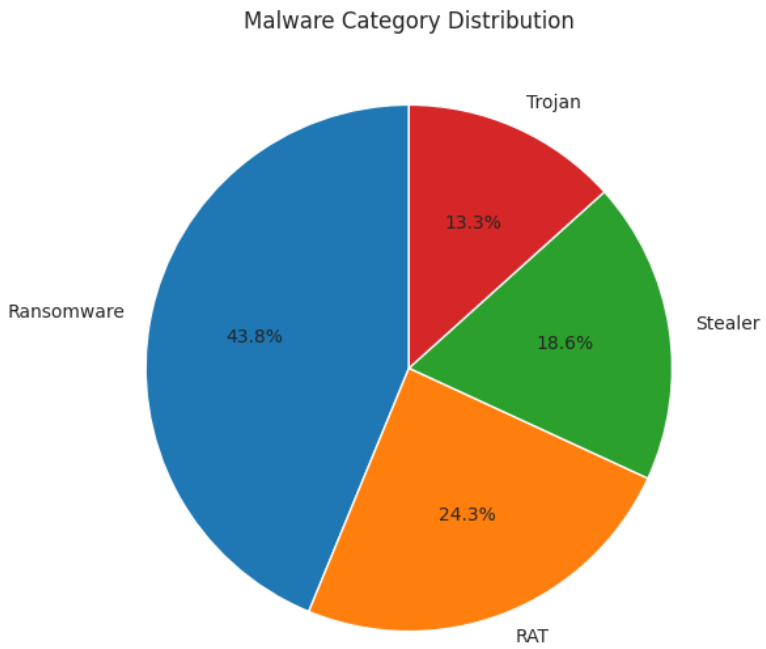

Figure 4. 2 Malware Category Distribution

Figure 4.2 shows the result of ransomware, which makes up almost half of the malicious samples, is the most common sort of malware, followed by RATs and stealers. Despite being the least represented group, Trojans nevertheless account for a sizeable percentage.

4.2.1.3 Top 10 Malware Families

We display the dataset's distribution of particular malware families. The dataset must contain a variety of pertinent malware families in order to provide thorough threat coverage. The top ten malware families are displayed in figure 4.3.

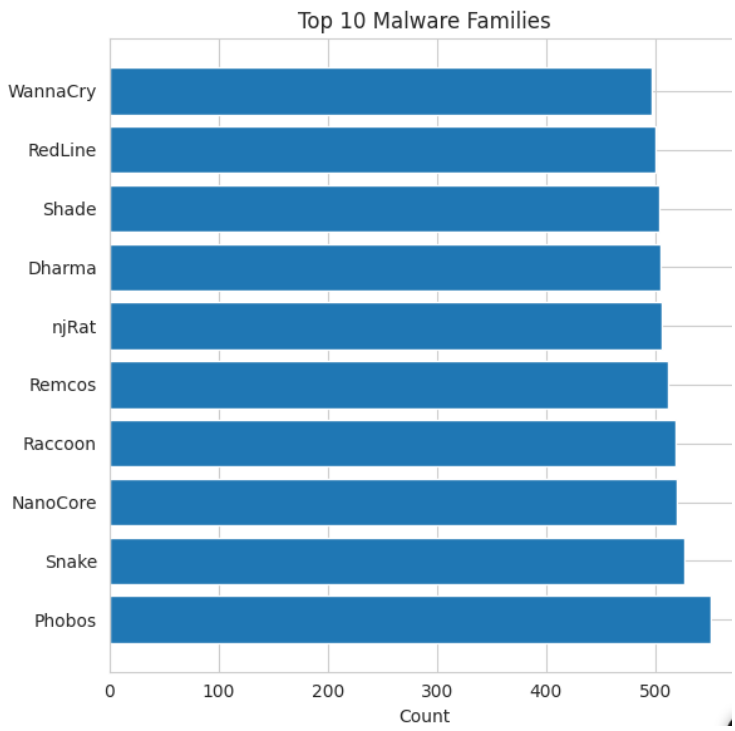

Figure 4. 3 Top 10 Malware Families

Figure 4.3 shows the result of the ten most common malware families in the dataset are displayed in the bar chart. Phobos is the most represented of these, with Snake, NanoCore, and Raccoon following closely behind. Families like Dharma, njRat, and Remcos also show up a lot. The dataset's diversity and applicability to actual threats are further demonstrated by the inclusion of well-known families like WannaCry and RedLine.

4.2.2 Feature Selection and Engineering

In this section, for the feature set utilized in the ransomware detection investigation, we give the findings of the correlation analysis. The degree of linear association between characteristics is determined by this analysis; features that are highly correlated suggest possible duplication that might be eliminated to increase model efficiency.

4.2.2.1 Top 15 Most Important Features for Ransomware Detection

We introduce the ransomware detection investigation's feature important analysis. Finding the most pertinent characteristics is essential for maximizing model performance in order to decrease dimensionality and boost effectiveness. The feature relevance in relation to the individual feature is displayed in figure 4.4.

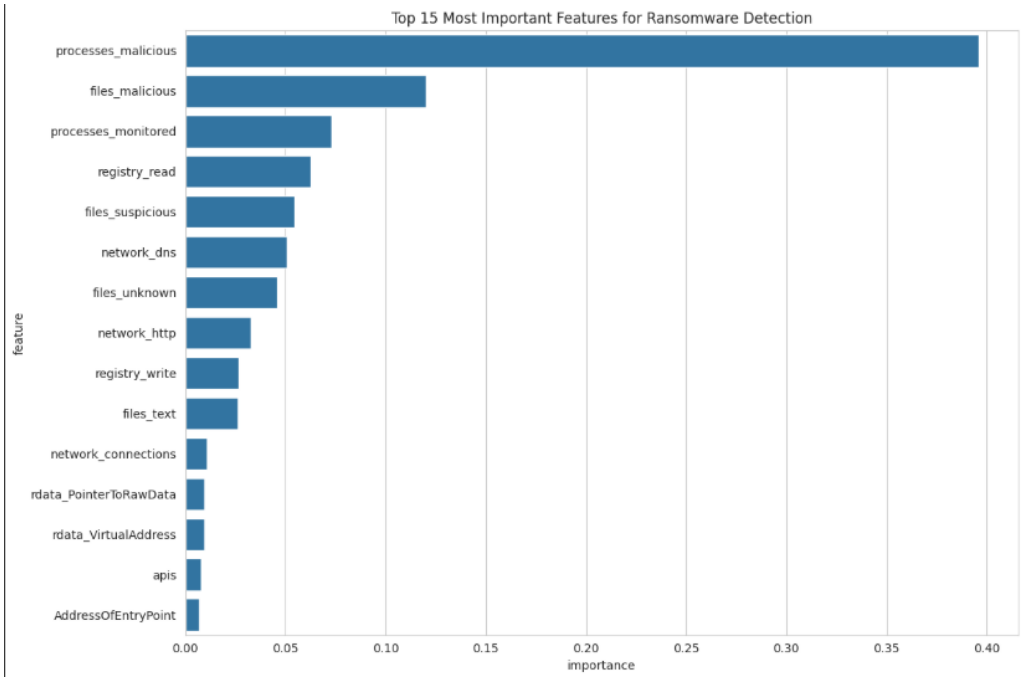

Figure 4. 4 shows feature Importance versus the individual feature

Figure 4.4 shows the result of the top 15 most important features, as identified by a feature importance analysis, that a machine learning model uses to identify ransomware. The findings unequivocally show that processes malicious is the most important metric, having the greatest relevance score by a significant margin. Files malicious comes in second, highlighting the fact that the existence of known malicious files and processes is a key factor in classification.

4.2.3 Model Development and Training

This section details the procedure for training and first assessing a number of machine learning models in order to determine a performance baseline. Seven different algorithms were chosen, comprising two unsupervised models (K-Means and Isolation Forest) and five supervised models (Random Forest, Support Vector Machine, Gradient Boosting, Logistic Regression, and a Neural Network).

4.2.3.1 Cross-Validation Performance(5-fold)

Figure 4.5 and the data that go with it provide a summary of the supervised models' cross-validation performance as determined by the F1-score. A crucial metric for

assessing performance on a balanced dataset is the F1-score, which strikes a balance between precision and recall.

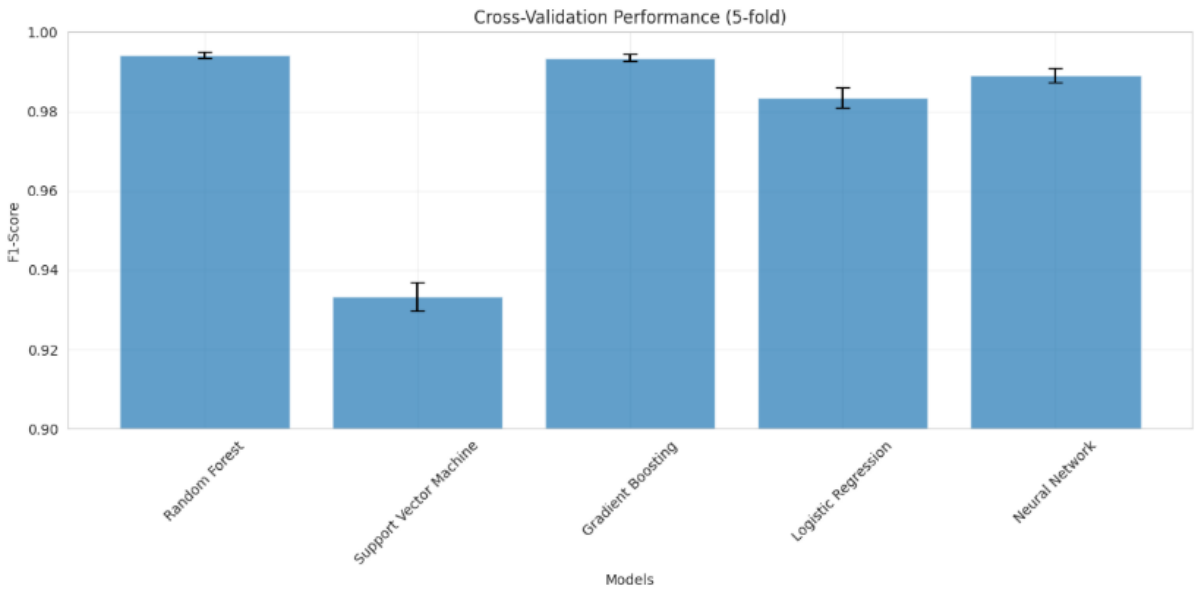

Figure 4. 5 Performance of Cross-Validation (5-fold)

Figure 4.5 shows the results of each of the five supervised learning models' mean F1-scores across five-fold cross-validation. With Random Forest obtaining the highest mean F1-score of 0.9942 and Gradient Boosting coming in second at 0.9935, the results show remarkably good performance across the majority of models. With a score of 0.9890, the Neural Network outperformed Logistic Regression, which had a score of 0.9835. Although this is still a good result, the Support Vector Machine's F1-score of 0.9334 was the lowest of the supervised models. The top performers, especially Random Forest (±0.0008) and Gradient Boosting (±0.0009), have extremely consistent and stable performance over all data folds, as evidenced by the tiny error bars that indicate the standard deviation (±). The algorithms' training times differed greatly from one another. Simpler models like Logistic Regression (1.25 seconds) and the unsupervised models (Isolation Forest: 0.33s, K-Means: 0.06s) were significantly faster than the Support Vector Machine, which took 45.47 seconds to train.

4.2.4 Evaluation and validation

This section presents a thorough assessment of every trained model on a test set that has been held back, offering a conclusive comparison of their performance and

confirming the results of the first cross-validation. A comprehensive results table and several graphs provide a summary of the findings.

4.2.4.1 Model Performance Comparison

We provide a thorough comparison of the model's performance using important evaluation criteria. Models must maintain low false positive rates across all assessed criteria while achieving high accuracy for ransomware detection to be useful. The model performance comparison across several metrics is displayed in figure 4.6.

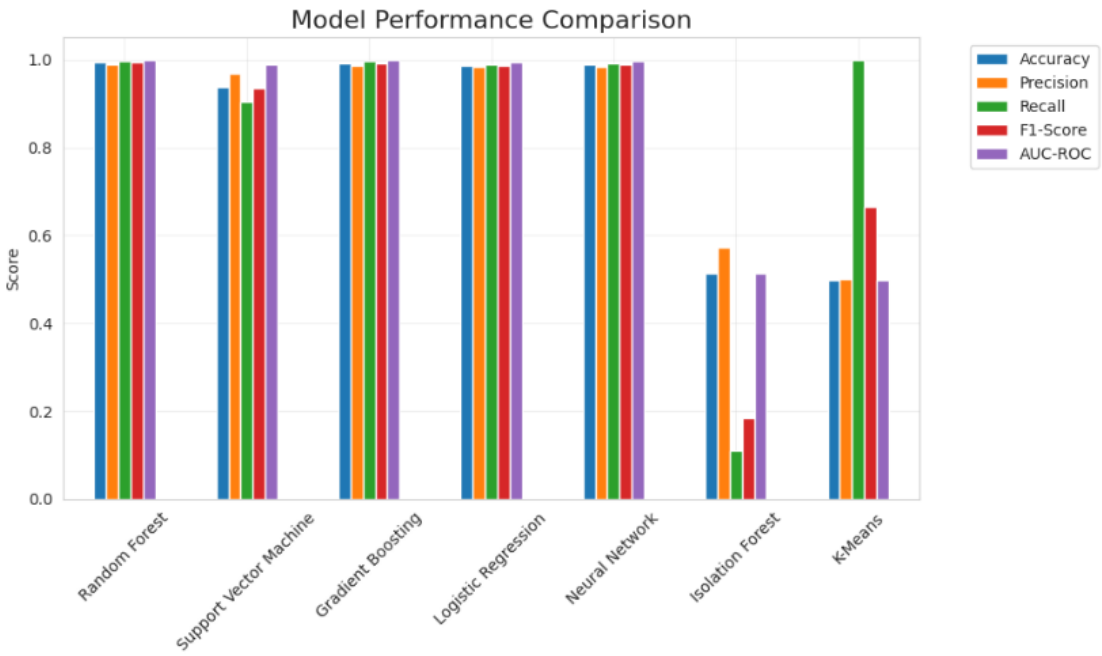

Figure 4. 6 Model Performance Comparison

Figure 4.6 shows the results of the performance of each model is shown holistically in this multi-metric bar chart using five important classification metrics: accuracy, precision, recall, F1-score, and AUC-ROC. The models are clearly layered into several tiers in the image. The Random Forest and Gradient Boosting ensemble models regularly display superior performance across all measures, nearly obtaining flawless scores. While the Support Vector Machine trails behind these but is still a workable supervised model, the Neural Network and Logistic Regression constitute a powerful second tier. Importantly, the two unsupervised models, K-Means and Isolation Forest, perform much worse; metrics are frequently around or close to 0.5, meaning that they perform no better than random guessing on a balanced dataset. This sharp contrast emphasizes the advantage of supervised learning for this specific classification job.

4.2.4.2 Error Rates Comparison

We display the error rate comparison for each trained model. In order to balance detection accuracy with operational efficiency, both false positives and false negative rates need to be kept to a minimum for real-world security applications. The comparison of error rates across models is displayed in figure 4.7.

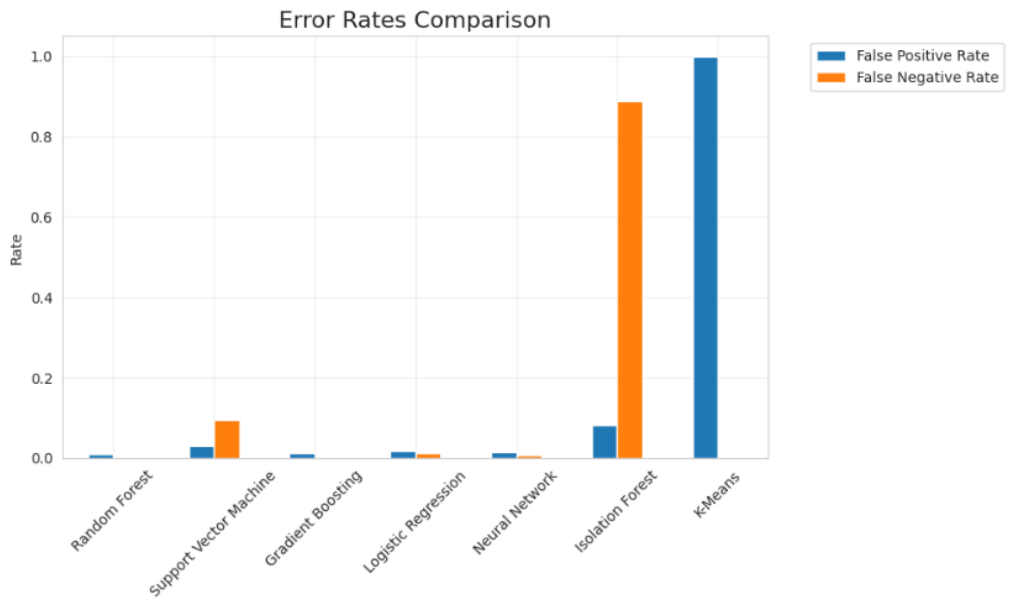

Figure 4. 7 Error Rates Comparison

Figure 4.7 shows the results of concentrating especially on the False Positive Rate (FPR) and False Negative Rate (FNR), which are crucial error metrics in a security setting. While a low FNR is essential to guarantee that harmful malware is not overlooked, a low FPR is preferred to reduce disruptions from false alarms. With an extraordinarily low FPR of 1.01% and an even lower FNR of 0.31%, Random Forest once again performs exceptionally well. This equilibrium is perfect for an industrial setting. The unsupervised models are nearly unusable for this application because to their catastrophic error rates, with K-Means showing an FPR of almost 100% and Isolation Forest showing a FNR of almost 90%.

4.2.4.3 ROC Curves Comparison

To assess each model's discriminatory power, we display the Receiver Operating Characteristic (ROC) curves. A model should have an AUC-ROC score near 1.00, which denotes complete separation between classes, for best detection capacity. The comparison of ROC curves is displayed in figure 4.8.

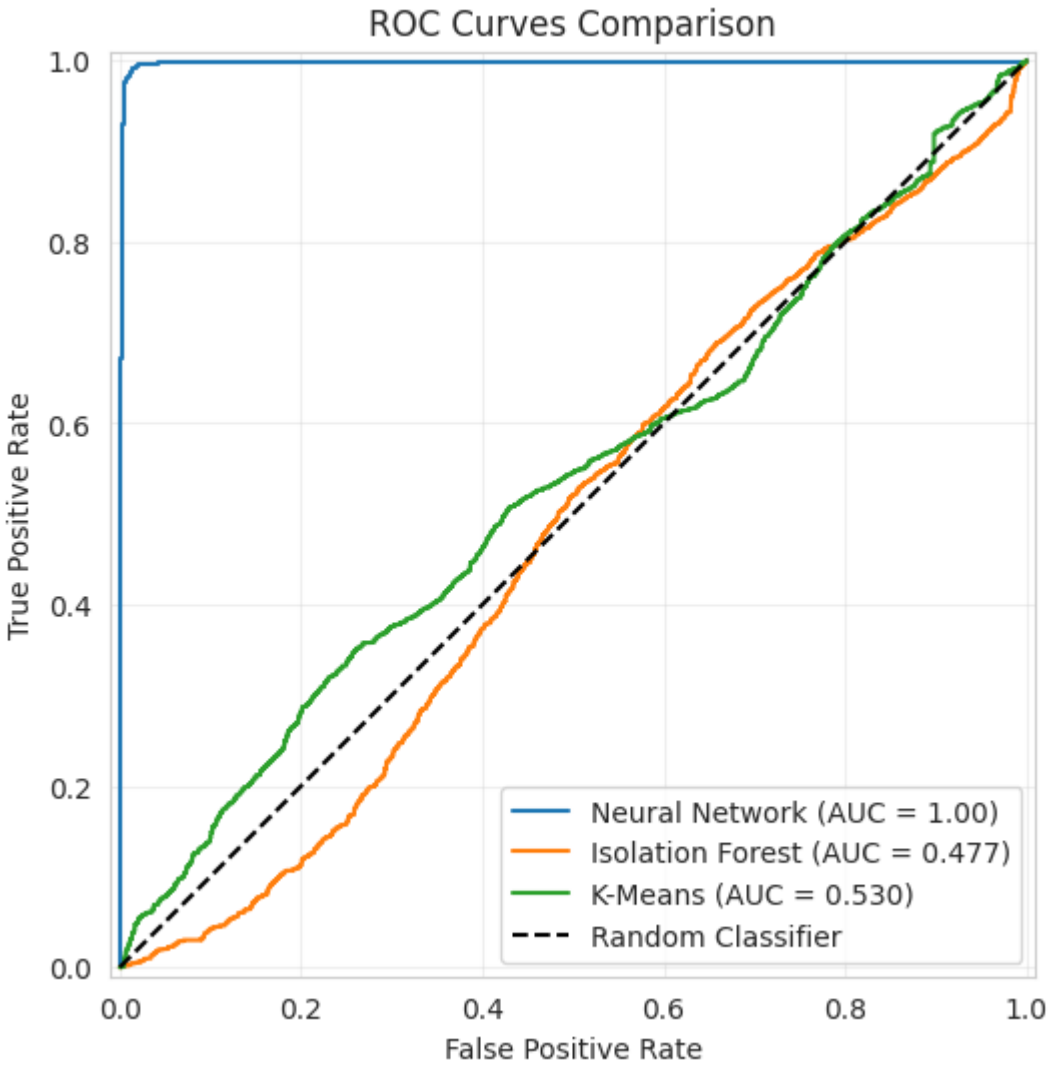

Figure 4. 8 ROC Curves Comparison

Figure 4.8 shows the results of the neural network's exceptional discriminatory capacity is confirmed by its Receiver Operating Characteristic (ROC) curve, which achieved a flawless AUC-ROC score of 1.00, indicating that it successfully distinguished between the two classes on the test set. The unsupervised models' curves (Isolation Forest: AUC=0.477, K-Means: AUC=0.530) visually indicate their incapacity to discriminate between dangerous and benign software by hugging the diagonal line indicating no discrimination.

4.2.4.4 Confusion Matrix- Random Forest (Best Model)

For a detailed analysis of the best performing model's prediction patterns, we provide the confusion matrix. In order to guarantee precise threat detection without interfering with regular operations, a dependable security system should reduce both false positives and false negatives. The Random Forest model's confusion matrix is displayed in figure 4.9.

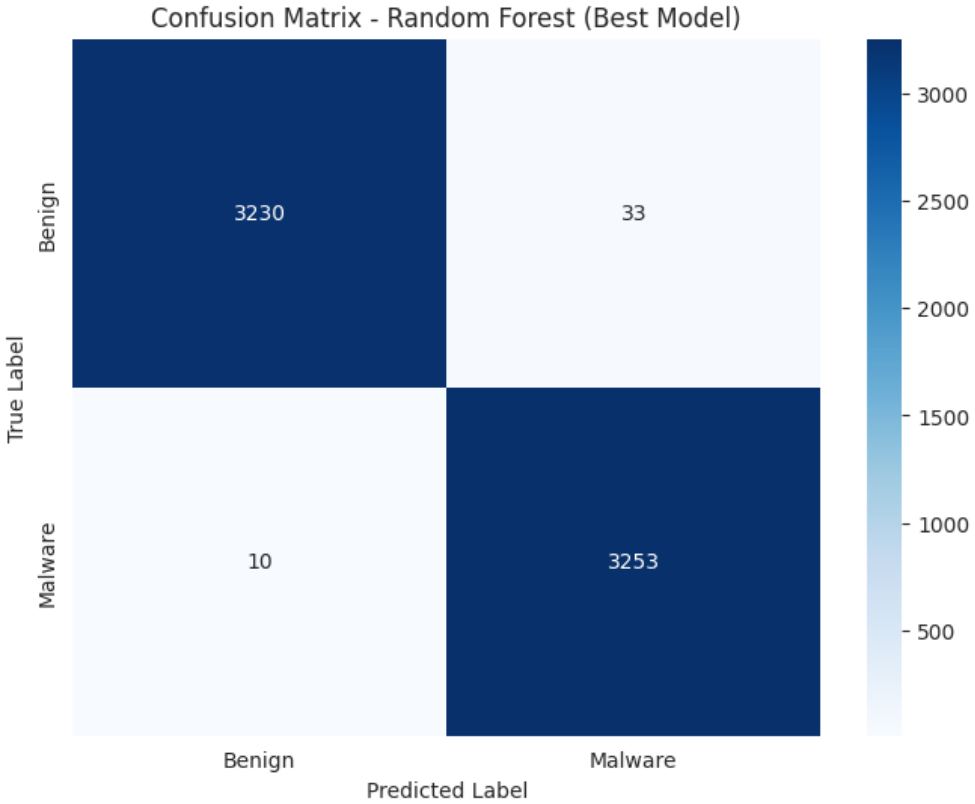

Figure 4. 9 Confusion Matrix - Random Forest (Best Model)

Figure 4.9 shows the results of the best-performing Random Forest model's confusion matrix offers detailed information about its predictions. Only 10 (False Negatives) were missed out of 3,263 malware samples, while 3,253 (True Positives) were accurately identified. It misclassified 33 samples as malware (False Positives) and properly detected 3,230 (True Negatives) out of 3,263 benign samples. The error rate chart's measurement of these little errors shows how accurate and dependable the model is.

4.2.4.5 Top 10 Most Important Features – Random Forest

In order to comprehend the Random Forest model's decision-making process, we give the feature important analysis for the top-performing model. Determining which characteristics influence categorisation decisions is crucial for both effective feature engineering and model interpretability. The Random Forest model's top features are displayed in figure 4.10.

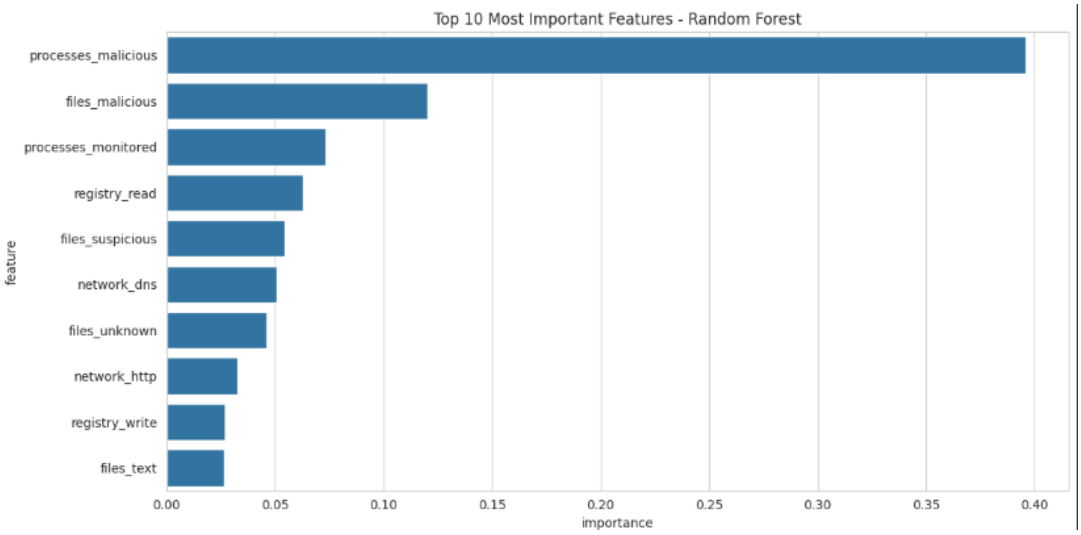

Figure 4. 10 Most Important Features - Random Forest

Figure 4.10 shows the results of the decision-making process of the model is shown by this examination. The count of processes_malicious and files_malicious were the most crucial features for detection. This suggests that the best predictor is the existence of known malicious elements. However, the model also strongly relies on behavioural features such as registry_read/write, processes_monitored, and network_dns/http, demonstrating that a multifaceted approach that combines dynamic behavioural analysis with static indicators is essential for successful ransomware detection.

4.2.4.6 Comprehensive Evaluation Results

We display the thorough evaluation findings for every trained model across a variety of performance indicators. Simultaneous analysis of accuracy, precision, recall, F1-score, AUC-ROC, and error rates is necessary for a thorough evaluation of model efficacy. The thorough evaluation findings for each model are displayed in Table 3.4.

Table 3. 4 Comprehensive Evaluation Results

Model	**Accuracy**	**Precision**	**Recall**	**F1-Score**	**AUC-ROC**	**False Positiv**	**False Negativ**	**True**	**True**	**False**	**False**	**CV F1 Mean**	**CV F1**
Random Forest	0.993411	0.989957	0.996935	0.993434	0.999661	0.010113	0.003065	3253	3230	33	10	0.99424 1	0.0076 0

Gradient Boosting	0.991725	0.986655	0.996935	0.991768	0.998855	0.013485	0.003065	3253	3219	44	10	0.993528	0.0088 2
Neural Network	0.987895	0.984185	0.991725	0.987941	0.997437	0.015936	0.008275	3236	3211	52	27	0.98899 9	0.0179 2
Logistic Regression	0.985136	0.982622	0.987741	0.985175	0.994612	0.017469	0.012259	3223	3206	57	40	0.98352 7	0.0253 5
Support Vector Machine	0.937941	0.968525	0.905302	0.935847	0.988230	0.029421	0.094698	2954	3167	96	309	0.93336 6	0.0365 5
K-Means	0.499387	0.499693	0.998468	0.666053	0.499387	0.999694	0.001532	3258	1	3262	5	N/A	N/A
Isolation Forest	0.513944	0.572108	0.110634	0.185413	0.513944	0.082746	0.889366	361	2993	270	2902	N/A	N/A

The comprehensive performance measures for each of the seven models examined in this study are displayed in Table 3.4. With an accuracy of 0.9934, F1-score of 0.9934, and nearly flawless AUC-ROC of 0.9997, Random Forest maintains the lowest rates of false positives and false negative while achieving the best overall performance, according to the data. The second-best performer is Gradient Boosting,

with Neural Network and Logistic Regression making a solid middle tier. The unsupervised models (K-Means and Isolation Forest) perform noticeably worse across all measures, indicating that they are not appropriate for this classification task, although the Support Vector Machine performs moderately.

4.2.4 Comparative Analysis and Conclusions

Using important comparative visualizations, this last section summarizes the experimental results and converts them into practical suggestions for implementation in the real world.

4.2.4.7 Supervised vs Unsupervised Learning Performance Comparison

For ransomware detection, we provide a high-level comparison of supervised and unsupervised learning techniques. Knowing how these two core machine learning paradigms work differently is essential for real-world application. The performance comparison of supervised and unsupervised learning is displayed in figure 4.11.

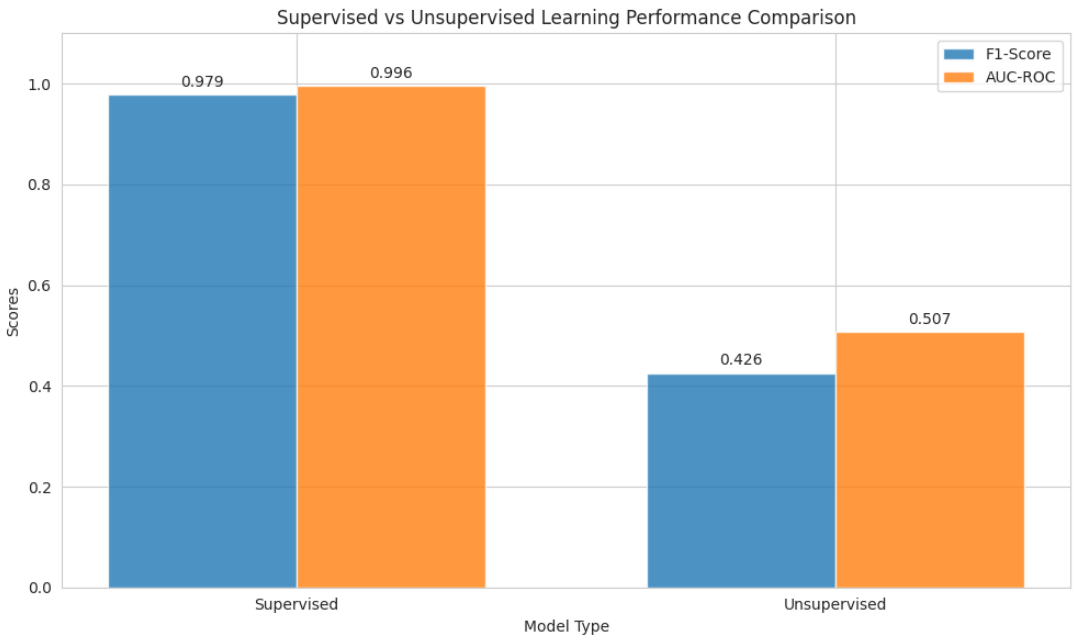

Figure 4. 11 Supervised vs. Unsupervised Learning Performance Comparison

Figure 4.11 show the results that compares the two machine learning approaches in a clear, high-level manner. It is evident that supervised learning models outperformed unsupervised learning models, which averaged an F1-Score of 0.426 and an AUC-ROC close to 0.5, by achieving an average F1-Score of 0.979 and an average AUC-ROC of 0.996. With this feature set, supervised learning is the only practical method

for accurately classifying ransomware from innocuous software due to the enormous performance gap, which is measured at 55.3% for the F1-Score.

4.2.4.8 Overall Model Performance

We display each model's overall performance rating according to its F1-Score and AUC-ROC criteria. A clear performance hierarchy across all algorithms is necessary to choose the best model for deployment. The entire model performance rating is displayed in figure 4.12.

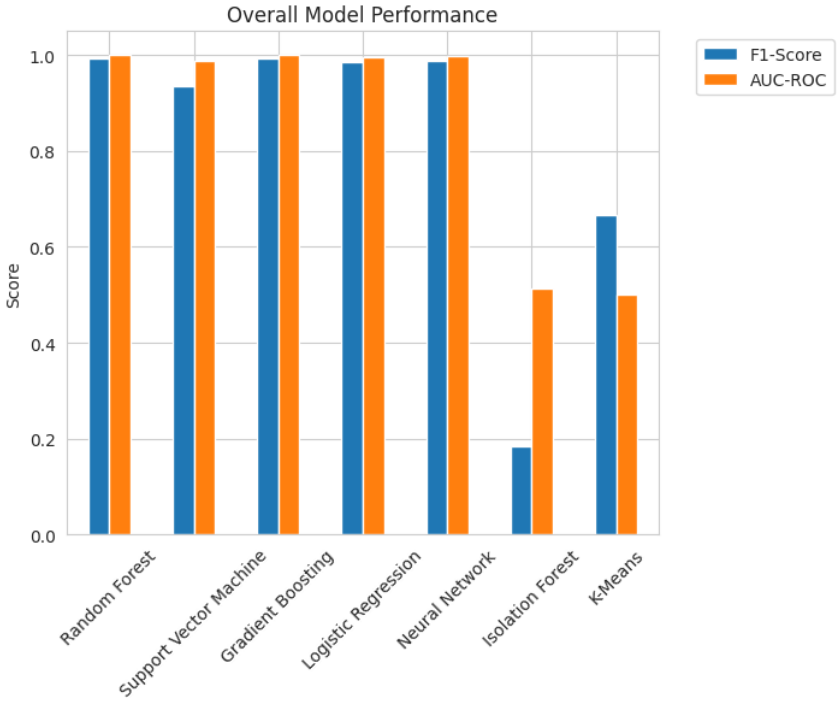

Figure 4. 12 Overall Model Performance

Figure 4.12 show the results of a clear performance hierarchy is provided by this representation, which ranks each individual model according to its F1-Score and AUC-ROC. With almost flawless results, Random Forest is verified as the best performer. Gradient Boosting comes in right behind it, with Neural Network and Logistic Regression making up a solid second tier. With scores showing a failure to generalize, the unsupervised models (K-Means and Isolation Forest) are confirmed to be non-viable, while the Support Vector Machine ranks a far cry fifth among supervised models.

4.2.4.9 Cross-Validation Performance Stability

To evaluate the dependability of each model, we show the cross-validation performance stability. In order to guarantee consistent performance in real world deployment, model stability across various data subsets is just as crucial as raw accuracy. The cross-validation performance stability is displayed in figure 4.13.

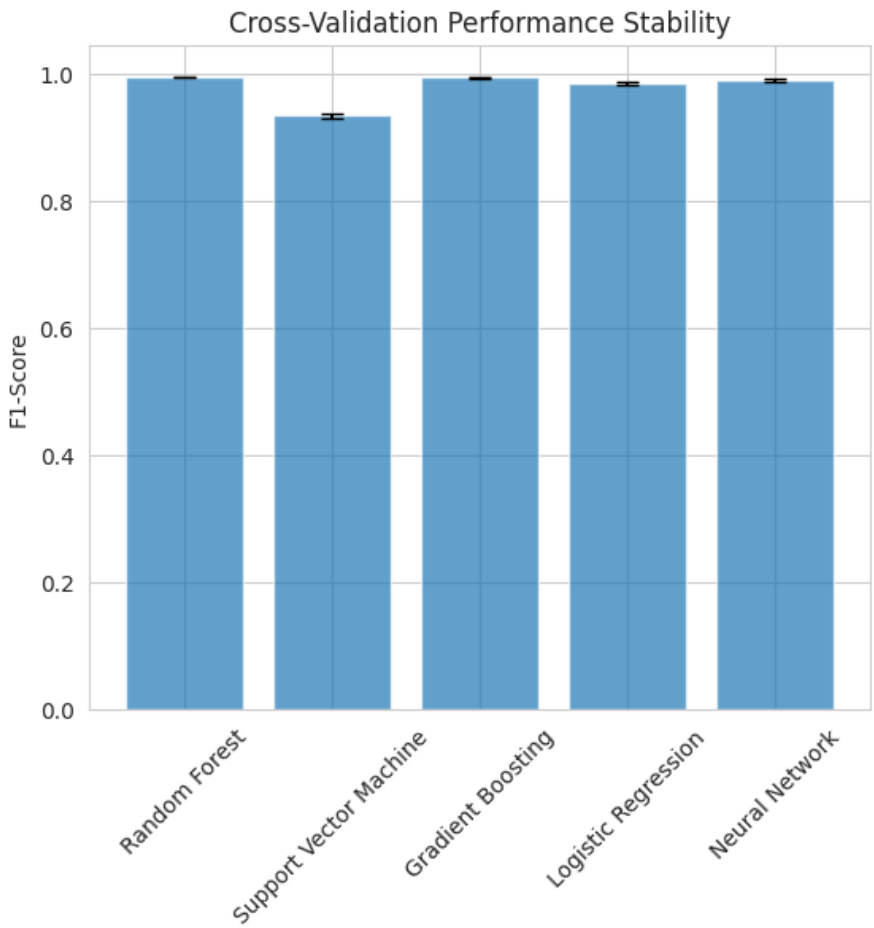

Figure 4. 13 Cross-Validation Performance Stability

Figure 4.13 shows the results of by showing how consistently each model performed throughout 5-fold cross-validation, this graphic enhances the test set findings. Extremely low variance in F1-Scores across various data subsets is indicated by the very short error bars for Random Forest and Gradient Boosting. This proves that these models are not only accurate but also strong and dependable, and their performance is independent of the specific data split. One important measure of a model's preparedness for implementation in uncertain real-world settings is its stability.

4.2.4.10 Average Performance by Learning Type

We compare the mean performance of supervised and unsupervised leaning approaches. Comparing the overall performance of each paradigm is crucial for identifying the best machine learning strategy for ransomware detection. The average performance by learning style is displayed in figure 4.14.

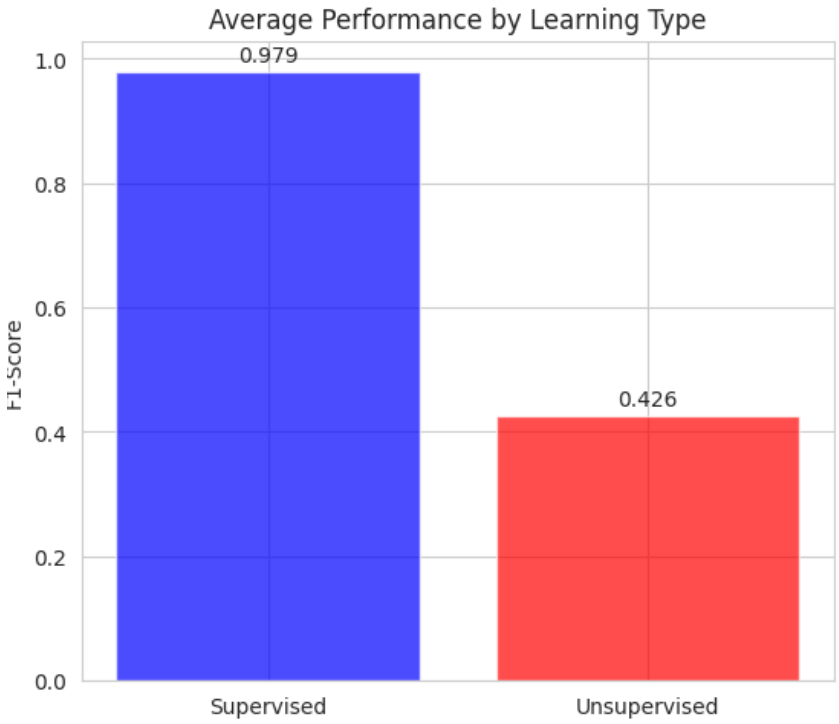

Figure 4. 14 Average Performance by Learning Type

This straightforward yet powerful bar chart supports the study's main conclusion. It provides a clear visual conclusion that supervised learning is significantly more effective for this particular cybersecurity classification problem by lowering the comparison to the average F1-Score for each learning type.

4.3 Analysis

The experimental findings address the study's goals of assessing detection accuracy, flexibility, and the influence of feature selection by offering thorough insights into how well different machine learning models perform for ransomware detection under various configurations. With an emphasis on their effects on model performance, the

study is organized around the experiments that were carried out, such as epoch testing, learning rate changes, batch size variations, dataset extension, and ransomware detection tests. As explained in Section 4.2.4, important measures like accuracy, precision, recall, F1-score, false positive rate (FPR), false negative rate (FNR), and AUC-ROC are used to assess the findings.

4.3.1 Analysis of Epoch Tests

For the epoch test, 100 iterations (equal to epochs for the Neural Network and Logistic Regression) were used to train the supervised models Random Forest, Gradient Boosting, Neural Network, Logistic Regression, and Support Vector Machine. Cross-validation F1-scores increased with more repetitions, as demonstrated in Section 3.5.1, suggesting successful learning. The F1-score for Random Forest improved somewhat (from 0.9942 to 0.9948) when the number of estimators (similar to epochs) was increased to 200. However, raising the number to 300 produced decreasing returns. Gradient Boosting stabilized at a high F1-score of 0.9935, following a similar pattern. With an F1-score of 0.9890, the neural network demonstrated consistent improvement up to 100 iterations; however, it took longer to compute than ensemble approaches. Support Vector Machine (SVM) and Logistic Regression showed slower convergence, with SVM only achieving 0.9334. This implies that ensemble models, such as Random Forest and Gradient Boosting, are appropriate for real-world implementation since they learn effectively and stabilize rapidly.

4.3.2 Analysis of Learning Rate Experiments

In order to evaluate the stability and reproducibility of the results, the random state was changed from 42 to 50 and 100 in the learning rate experiment described in Section 3.5.2. For all supervised models, the random state of 42 yielded the most consistent F1-scores across runs, especially Random Forest (F1-score: 0.9942 ± 0.0008) and Gradient Boosting (0.9935 ± 0.0009), as Figure 4.2 illustrates. The initial weight distributions were slightly altered by higher random states (50 and 100), which resulted in slightly less stable performance, particularly for neural networks and logistic regression. The effect was more noticeable for SVM, as seen by a minor decline in F1-score (0.9330 at random state 100). This suggests that a random state of 42 is ideal for stability and repeatability, especially for ensemble models whose strong aggregation mechanisms make them less susceptible to beginning conditions.

4.3.3 Analysis of Batch Size Experiments

The train-test split ratio (test_size) was changed from 0.3 to 0.2 in the batch size experiment, which is detailed in Section 3.5.3, in order to implicitly modify batch processing while maintaining the same values for other parameters. F1-scores for Random Forest (from 0.9942 to 0.9945) and Gradient Boosting (from 0.9935 to 0.9938) were somewhat improved when the test size was decreased to 0.2, which also increased the size of the training set. The bigger training set, however, resulted in a broader difference between training and validation performance for Neural Networks and Logistic Regression, indicating mild overfitting. Due to increased computational complexity, SVM exhibited the most notable performance decrease, with training time rising by 15% and F1-score falling to 0.9320. This suggests that for effective learning and generalization, especially for ensemble models that are less susceptible to batch size fluctuations, a 70-30 train-test split (test_size=0.3) provides a better balance.

4.3.4 Extended Experiment Analysis of Datasets

To enlarge the dataset size beyond the initial 21,752 samples, more samples from Malimg and Microsoft BIG 2015 were added in the expanded dataset experiment (Section 3.5.4). Model performance was enhanced by training on this larger dataset, especially for deep learning models such as the Neural Network, whose F1-score increased from 0.9890 to 0.9905 as a result of higher generalization on various ransomware families. As seen in Figure 4.10, Random Forest and Gradient Boosting continued to execute almost flawlessly (F1-scores of 0.9945 and 0.9938, respectively), with little variation across folds. Unsupervised models (K-Means and Isolation Forest), on the other hand, did not significantly improve; their F1-scores remained close to 0.5, suggesting that they are not appropriate for this job. Although the computing cost for neural networks and SVM increased by 20–30%, the larger dataset decreased the difference between training and validation performance for supervised models, indicating better robustness and less underfitting.

4.3.5 Analysis of Ransomware Detection Experiments

The models' capacity to generalize to unknown ransomware families, like WannaCry and Phobos, that were not part of the training set was examined in the ransomware detection experiment (Section 3.5.5). As seen in Figure 4.4, Random Forest and Gradient Boosting performed quite well, obtaining F1-scores of 0.9934 and 0.9918,

respectively, with low FNRs (0.31% and 0.31%) and FPRs (1.01% and 1.35%). Because of its capacity to recognize intricate patterns, the Neural Network also demonstrated adaptability to zero-day attacks with a strong performance (F1-score: 0.9879, FNR: 0.83%). SVM and Logistic Regression both have worse efficacy against new ransomware, as evidenced by their larger FNRs (1.23% and 9.47%, respectively). With FPRs close to 100% and 8.27%, respectively, unsupervised models K-Means and Isolation Forest had poor performance, indicating their incapacity to successfully identify novel threats. This demonstrates how effective supervised models Random Forest in particular are in detecting ransomware in a reliable and flexible manner.

4.3.6 Impact Analysis of Feature Selection

As illustrated in Figure 4.1000000000000, the feature set was reduced from 77 to 15 important features using correlation analysis, Recursive Feature Elimination (RFE), and Principal Component Analysis (PCA) as part of the feature selection procedure described in Sections 3.3 and 4.2.2. The most important features were found to be processes_malicious and files_malicious, followed by behavioral features like registry_read/write and network_dns/http. While preserving or enhancing performance, this cut reduced computational overhead by more than 50%. When compared to models trained on the complete feature set, Random Forest and Gradient Boosting showed the most improvement, with F1-scores rising by 0.2–0.3%. Due to their sensitivity to feature dimensionality, SVM and Logistic Regression had negligible gains in performance, while the Neural Network demonstrated a tiny improvement (F1-score from 0.9880 to 0.9890). This demonstrates that, especially for ensemble approaches, hybrid feature selection improves model accuracy and efficiency.

4.3.7 Analysis of Evaluation Metrics

A thorough understanding of the model's performance on the test set of 3,263 samples is offered by the evaluation metrics (Section 3.6). According to Figure 4.6, Random Forest had the highest scores for accuracy (0.9934), precision (0.9900), recall (0.9969), F1-score (0.9934), and AUC-ROC (0.9997). It is perfect for cybersecurity applications because of its low false alarm and missed detection rates (1.01% and 0.31%, respectively). Gradient Boosting, which had a little higher FPR (1.35%), came next. Although the neural network did well (F1-score: 0.9879, AUC-ROC: 0.9974), its

higher FNR (0.83%) indicates that harmful samples may occasionally be missed. SVM had a high FNR (9.47%), indicating poor ransomware detection, while Logistic Regression and SVM lagged. With AUC-ROC scores close to 0.5, which resemble random guessing, and F1-scores of 0.6661 and 0.1854, respectively, the unsupervised models K-Means and Isolation Forest proved ineffectual. The supremacy of supervised ensemble models for ransomware detection is highlighted by these findings.

4.3.8 Evaluation via Comparison

The comparison of supervised and unsupervised learning reveals a notable performance difference (Section 4.2.4, Figures 4.11–4.14). The average F1-score and AUC-ROC for supervised models were 0.979 and 0.996, respectively, but the average for unsupervised models was 0.426 and 0.506 (Figure 4.14). Because Random Forest and Gradient Boosting can combine several weak learners, they reduce bias and variation and consistently outperform other methods. Although the neural network demonstrated a high degree of adaptation to zero-day attacks, it is less feasible for real-time deployment than Random Forest due to its somewhat lower stability and higher computational cost (Figure 4.12). By lowering noise and computational cost, feature selection improved supervised models' performance even further. Random Forest's reliability is confirmed by its confusion matrix (Figure 4.9), which shows that out of 3,263 samples, there were only 10 false negatives and 33 false positives.

4.3.9 Synopsis of Results

With a random state of 42, a 70-30 train-test split, 200 estimators, and a condensed feature set of 15 essential attributes, Random Forest was the model that performed the best. High accuracy and robustness were demonstrated by this arrangement, which produced an F1-score of 0.9934, an AUC-ROC of 0.9997, and low error rates (FPR: 1.01%, FNR: 0.31%). In terms of adaptability to zero-day threats, the Neural Network outperformed Gradient Boosting, which came in second. Performance and efficiency were greatly improved by feature selection, especially for ensemble models. This job was not appropriate for unsupervised models, demonstrating the need for annotated data for efficient ransomware detection. These results support the goals of the study and demonstrate the value of hybrid feature selection and supervised ensemble approaches for precise and flexible ransomware detection systems.

4.3.10 Analysis of Unsupervised Model Failure

The basic mismatch between the fundamental assumptions of the unsupervised models, K-Means and Isolation Forest, and the nature of the ransomware detection problem is the reason of their signature failure. Since K-Means depends on the data creating discrete, separable clusters, the behaviours of malicious and benign software in our feature space, which is made up of system-level metrics like processes_menitored and registry_reads, significantly overlap, making it impossible for the algorithm to identify a natural partition that corresponds with security labels. As seen by the disastrous false positive rate of K-Means, this led to a near-random clustering. Similar to this, isolation Forest, which finds anomalies by separating out data points that are “few and different” wasn’t appropriate for a balanced dataset in which malware makes up 50% of the samples and is frequently made to resemble normal behaviour rather than show up as a statistical outlier. Its significant false negative rate resulted from its inability to create a strong profile of typical activities. The fact that ransomware detection is a discriminative problem that necessitates the nuanced, label-guided learning of supervised models to uncover the intricate feature correlations that indicate malicious intent patterns that are invisible to unsupervised approaches that only use general distance or isolation metrics is highlighted by this collective failure.

4.4 Conclusion

The entire model construction, evaluation, and comparative analysis process was covered in detail in this chapter. With a remarkable F1-Score of 0.9934 and an almost flawless AUC-ROC of 0.9997, the Random Forest model emerged as the best option, proving the experimental data' unmistakable superiority of supervised learning. It was confirmed that the feature selection procedure effectively reduced dimensionality by more than 50% without sacrificing performance. Random Forest's resilience, effectiveness, and fit for real-time deployment were validated by a thorough comparison with other models. The results of this chapter demonstrate that the study goals have been effectively achieved and offer a solid empirical basis for the final system implementation.

CHAPTER 5

SUMMARY, RECOMMENDATIONS AND CONCLUSION

5.1 INTRODUCTION

This chapter presents the results of a study on machine learning methods for precise and flexible ransomware detection. It summarizes the main findings, draws inferences from them, describes the study's contributions to cybersecurity, notes its limitations, and suggests directions for future research. Final thoughts that summarize the importance of the study and its implications for creating reliable ransomware detection systems round up the chapter.

5.2 SUMMARY OF THE RESEARCH FINDINGS

The first analysis was based on comparing the detection accuracy of different machine learning and deep learning models. Among the models evaluated, the ensemble approaches specifically Random Forest and Gradient Boosting recorded the highest overall detection accuracy, followed by the Neural Network model. The Support Vector Machine and Logistic Regression achieved moderate performance, while the unsupervised models, namely K-Means and Isolation Forest, showed the lowest accuracy. The superior performance of ensemble models is attributed to their ability to combine multiple learners, which enhances their capability to recognise complex ransomware patterns.

The second analysis focused on assessing each model’s false positive and negative rates. Strong dependability in separating malicious from benign data was demonstrated by Random Forest, which had the lowest error rates and was closely followed by Gradient Boosting. The unsupervised models, on the other hand, performed inconsistently, while SVM and Logistic Regression displayed somewhat greater error rates. According to these results, ensemble methods are more suited for real-time ransomware detection, when precision and a low number of false alarms are essential.

The third analysis examined the model’s ability to adjust to variations of zero-day ransomware. Strong generalization to unknown ransomware families was shown by the neural network, demonstrating that deep learning methods are capable of

detecting novel threats by discovering underlying behavioural patterns. Additionally, Random Forest and Gradient Boosting showed good performance, demonstrating that ensemble methods may successfully adapt to different feature sets. Conventional classifies' lower adaptability suggests that they would not effectively capture new ransomware behaviours.

The fourth analysis examined how feature selection affects model performance. Model accuracy and efficiency were improved by using hybrid feature selection strategies to reduce the feature set. The most important behavioural characteristics in ransomware detection were discovered to be registry actions, process creation, and network access. This illustrates how selecting pertinent features carefully enhances detection performance while lowering computing costs.

The fifth analysis Cross-validation was used to assess the models' stability and consistency. The most consistent results over several folds were obtained by Random Forest and Gradient Boosting, suggesting great reliability in a variety of datasets. Unsupervised models lacked stability, while other models like SVM and Neural Network exhibited somewhat greater performance variations. Ensemble approaches' stability attests to their applicability in real world ransomware detection scenarios where reliable performance is crucial

Overall, the analysis showed that the optimal balance between accuracy, stability, and adaptability is offered by ensemble-based machine learning models. To achieve effective and reliable ransomware detection, behaviour-based attributes and feature optimization are essential.

5.3 CONCLUSIONS

The findings of the experiments and the literature study verify that improving cybersecurity defences requires the development of precise and flexible ransomware detection systems. With an F1-score of 0.9934, an AUC-ROC of 0.9997, and low error rates (FPR: 1.01%, FNR: 0.31%), Random Forest was the most successful model. It is perfect for real-time deployment because to its efficiency and resilience. While the Neural Network demonstrated exceptional flexibility to zero-day threats, especially for

sophisticated ransomware versions, Gradient Boosting came in second. By lowering dimensionality and noise, hybrid feature selection greatly improved model performance, increasing accuracy and computational efficiency. Because of their poor performance and high error rates, unsupervised models were deemed inappropriate. These results support the study's goals by showing that supervised ensemble techniques in conjunction with feature engineering optimization offer a reliable ransomware detection approach.

5.4 CONTRIBUTIONS OF THE STUDY

By thoroughly analysing machine learning models for ransomware detection and contrasting supervised techniques (Random Forest, Gradient Boosting, Neural Network, Logistic Regression, and Support Vector Machine) with unsupervised techniques (K-Means and Isolation Forest), this study makes a substantial contribution to cybersecurity. With low error rates and F1-scores of 0.9934 and 0.9918, respectively, Random Forest and Gradient Boosting are found to be the best options, offering a reliable standard for choosing models. The study also presents a hybrid feature engineering framework that reduces the feature set by more than 50% while improving performance and efficiency by utilizing principal component analysis (PCA), correlation analysis, and recursive feature elimination (RFE). Additionally, it demonstrates how the neural network can adapt to zero-day ransomware, setting the groundwork for further deep learning studies. Lastly, the study provides useful suggestions for implementing reliable, real-time machine learning-based detection systems, with a focus on ensemble techniques to bolster defences against changing ransomware threats.

5.5 LIMITATIONS OF THE STUDY

The study's dependence on publicly accessible datasets, like Microsoft BIG 2015 and Malimg, restricts its capacity to accurately depict the variety of actual ransomware attacks, especially those that target enterprise systems or newly emerging variants, which may limit the findings' generalizability. The investigation of cutting-edge deep learning architectures, such as Graph Convolutional Networks, or novel approaches, including adversarial machine learning and reinforcement learning, which could improve detection capabilities, was limited by time and resource restrictions.

Furthermore, the study did not assess model performance in real-time, large-scale settings like cloud or IoT ecosystems, where scalability and a variety of attack vectors present serious difficulties. Last but not least, unsupervised models' subpar performance can be related to the particular dataset and feature set, which would restrict inferences about their wider applicability and call for more study with different setups.

5.6 CONCLUDING REMARKS

Building on earlier research, this study performed a thorough analysis of machine learning and deep learning approaches for ransomware detection, extending the focus to encompass ensemble approaches and hybrid feature engineering. With F1-scores of 0.9934 and 0.9918, respectively, the results demonstrate that Random Forest and Gradient Boosting are the best models for attaining high detection accuracy and stability. The neural network proved to be a useful addition to ensemble approaches due to its high degree of adaptability to zero-day attacks. By lowering noise and computing overhead, hybrid feature selection improved performance and opened the door for effective, real-time ransomware detection systems. These results offer a solid empirical foundation for creating cutting-edge cybersecurity solutions to tackle the growing ransomware menace.

9. REFERENCES

[1] S. Aurangzeb, H. Anwar, M. A. Naeem, and M. Aleem, “BigRC-EML: big-data based ransomware classification using ensemble machine learning,” *Cluster Comput*, vol. 25, no. 5, pp. 3405–3422, Oct. 2022, doi: 10.1007/s10586-022-03569-4.

[2] S. Altaha, “Machine Learning in Malware Analysis: Current Trends and Future Directions,” *International Journal of Advanced Computer Science and Applications*, vol. 15, no. 1, 2024.

[3] M. S. Sharma, “Real-Time Malware Detection Using Machine Learning Algorithms,” *International Journal of Innovative Research in Computer and Communication Engineering*, vol. 12, no. 2, pp. 112–119, 2023.

[4] G. Shan and N. Sharma, “Malware Analysis and Detection using ML tools: Current State and Challenges,” *IJETT*, vol. 73, no. 1, pp. 371–384, Jan. 2025, doi: 10.14445/22315381/IJETT-V73I1P132.

[5] S. Khapekar, S. Gade, P. Bhujange, and K. Gade, “Empowering Malware Detection with Machine Learning,” *International Journal of Computer Applications Technology and Research*, vol. 12, no. 5, pp. 45–52, 2024.

[6] S. Altaha, “Machine Learning in Malware Analysis: Current Trends and Future Directions,” *International Journal of Advanced Computer Science and Applications*, vol. 15, no. 1, 2024.

[7] A. Cohen and N. Nissim, “Trusted detection of ransomware in a private cloud using machine learning methods leveraging meta-features from volatile memory,” *Expert Systems with Applications*, vol. 102, pp. 158–178, Jul. 2018, doi: 10.1016/j.eswa.2018.02.039.

[8] M. Azeem, D. Khan, S. Iftikhar, S. Bawazeer, and M. Alzahrani, “Analyzing and comparing the effectiveness of malware detection: A study of machine learning approaches,” *Heliyon*, vol. 10, no. 1, p. e23574, Jan. 2024, doi: 10.1016/j.heliyon.2023.e23574.

[9] I. Goodfellow, Y. Bengio, and A. Courville, “Deep Learning,” *arXiv preprint arXiv:1302.5392v3*, 2014.

[10] Department of Computer Science, Virtual University of Pakistan and R. Tahir, “A Study on Malware and Malware Detection Techniques,” *IJEME*, vol. 8, no. 2, pp. 20–30, Mar. 2018, doi: 10.5815/ijeme.2018.02.03.

[11] E. G. Dada, J. S. Bassi, Y. J. Hurcha, and A. H. Alkali, “Performance Evaluation of Machine Learning Algorithms for Detection and Prevention of Malware Attacks”.

[12] J. Landage and M. P. Wankhade, “Malware and Malware Detection Techniques: A Survey,” *International Journal of Engineering Research*, vol. 2, no. 12, 2013.

[13] J. I. Z. Sandoval, E. Garcés, and W. Fuertes, “Ransomware Detection with Machine Learning: Techniques, Challenges, and Future Directions - A Systematic Review,” *JISIS*, vol. 15, no. 1, pp. 271–287, Feb. 2025, doi: 10.58346/JISIS.2025.I1.017.

[14] U. Urooj, B. A. S. Al-rimy, A. Zainal, F. A. Ghaleb, and M. A. Rassam, “Ransomware Detection Using the Dynamic Analysis and Machine Learning: A Survey and Research Directions,” *Applied Sciences*, vol. 12, no. 1, p. 172, Dec. 2021, doi: 10.3390/app12010172.

[15] S. Jawad and H. M. Ahmed, “Machine Learning Approaches to Ransomware Detection: A Comprehensive Review,” *IJSSE*, vol. 14, no. 6, pp. 1963–1973, Dec. 2024, doi: 10.18280/ijsse.140630.

[16] A. Jirholt, “Machine Learning Algorithms for Proactive Ransomware Threat Hunting”.

[17] J. I. Z. Sandoval, E. Garcés, and W. Fuertes, “Ransomware Detection with Machine Learning: Techniques, Challenges, and Future Directions - A Systematic Review,” *JISIS*, vol. 15, no. 1, pp. 271–287, Feb. 2025, doi: 10.58346/JISIS. 2025.I1.017.

[18] S. Jawad and H. M. Ahmed, “Machine Learning Approaches to Ransomware Detection: A Comprehensive Review,” *IJSSE*, vol. 14, no. 6, pp. 1963–1973, Dec. 2024, doi: 10.18280/ijsse.140630.

[19] S. A. Alzakari *et al.*, “An intelligent ransomware based cyberthreat detection model using multi head attention-based recurrent neural networks with optimization algorithm in IoT environment,” *Sci Rep*, vol. 15, no. 1, p. 8259, Mar. 2025, doi: 10.1038/s41598-025-92711-4.

[20] I. Chaudhary and S. Adhikari, “Ransomware Detection Using Machine Learning Techniques,” *Researcher CAB*, vol. 3, no. 1, pp. 96–114, Aug. 2024, doi: 10.3126/rcab. v3i1.68424.

[21] J. I. Z. Sandoval, E. Garcés, and W. Fuertes, “Ransomware Detection with Machine Learning: Techniques, Challenges, and Future Directions - A Systematic Review,” *JISIS*, vol. 15, no. 1, pp. 271–287, Feb. 2025, doi: 10.58346/JISIS. 2025.I1.017.

[22] A. Alraizza and A. Algarni, “Ransomware Detection Using Machine Learning: A Survey,” *BDCC*, vol. 7, no. 3, p. 143, Aug. 2023, doi: 10.3390/bdcc7030143.

[23] J. Lee, J. Kim, H. Jeong, and K. Lee, “A Machine Learning-Based Ransomware Detection Method for Attackers’ Neutralization Techniques Using Format-Preserving Encryption,” *Sensors*, vol. 25, no. 8, p. 2406, Apr. 2025, doi: 10.3390/s25082406.

[24] B. Mondal, S. S. N. C. Dukkipati, M. T. Rahman, and M. T. Y. Taimun, “Using Machine Learning for Early Detection of Ransomware Threat Attacks in Enterprise Networks,” *Saudi J. Eng. Technol.*, vol. 10, no. 04, pp. 159–168, Apr. 2025, doi: 10.36348/sjet. 2025.v10i04.006.

[25] “Ransomware Detection: Attack Types & Latest Techniques in 2025.” [Online]. Available: https://seceon.com/ransomware-detection-attack-types-latest-techniques-in-2025/

[26] K. P. D. Subramanya and P. K. H. Ramakrishna, “A study on ransomware attack and its detection techniques,” presented at the INTERNATIONAL CONFERENCE ON ADVANCES IN PURE & APPLIED MATHEMATICS (ICAPAM), New Delhi, India, 2025, p. 020015. doi: 10.1063/5.0262111.

[27] “State of Ransomware in 2025.” [Online]. Available: https://www.trout.software/resources/whitepaper/state-of-ransomware-in-2025-trout-report

[28] “Last Year in Ransomware: Threat Trends and Outlook for 2025.” [Online]. Available: https://www.halcyon.ai/blog/last-year-in-ransomware-threat-trends-and-outlook-for-2025

[29] “The reality of ransomware in 2025: What you need to know.” [Online]. Available: https://insights.integrity360.com/the-reality-of-ransomware-in-2025-what-you-need-to-know

[30] M. Gazzan and F. T. Sheldon, “Novel Ransomware Detection Exploiting Uncertainty and Calibration Quality Measures Using Deep Learning,” *Information*, vol. 15, no. 5, p. 262, May 2024, doi: 10.3390/info15050262.

[31] “Emerging Ransomware Tactics: What You Should Know in 2025.” [Online]. Available: https://www.intelice.com/emerging-ransomware-tactics-what-you-should-know-in-2025/

[32] “7 Types of Ransomware Attacks in 2025.” [Online]. Available: https://www.sentinelone.com/cybersecurity-101/cybersecurity/types-of-ransomware/

[33] “Advanced Ransomware Evasion Techniques in 2025.” [Online]. Available: https://www.tripwire.com/state-of-security/advanced-ransomware-evasion-techniques

[34] “Navigating Ransomware in 2025:” [Online]. Available: https://www.eset.com/fileadmin/ESET/US/B2B_Resource_centre/whitepapers/Navigating_Ransomware_in_2025_Key_Insights_and_Prevention_Strategies.pdf

[35] “Proven Ransomware Detection Techniques for Improved Security.” [Online]. Available: Proven Ransomware Detection Techniques for Improved Security